

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	2
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	6
ВВЕДЕНИЕ	9
1 АНАЛИЗ РЫНКА EDTECH И ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА В ОБРАЗОВАНИИ	12
1.1 Анализ российского рынка EdTech	12
1.2 Анализ мирового рынка EdTech	14
1.3 Анализ проблемы организации онлайн-обучения и потребностей преподавателей	15
1.4 Обзор технологий и решений в области AI-тьюторов и интеллектуальных образовательных систем	17
2 РАЗРАБОТКА И ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ MVP AI-ТЬЮТОРА	21
2.1 Дизайн эксперимента по разработке и оценке AI-тьютора	21
2.2 Обзор используемых технологий и инструментов	24
2.3 Реализация MVP ИИ-тьютора	26
2.4 Анализ результатов	32
3 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ	36
3.1 Архитектура продакшн решения	36
3.2 Разработка и имплементация решения MVP	42
3.3 Продуктовые направления развития	48
ЗАКЛЮЧЕНИЕ	52
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	57

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

AI (Artificial Intelligence) — искусственный интеллект, совокупность технологий и методов, позволяющих компьютерным системам выполнять задачи, требующие интеллектуальной деятельности человека.

EdTech (Education Technology) — отрасль цифровых технологий, направленная на разработку и внедрение технологических решений в сфере образования.

LLM (Large Language Models) — большие языковые модели, разновидность моделей машинного обучения, обученных на больших объемах текстовых данных и способных генерировать и анализировать текстовую информацию.

LMS (Learning Management System) — система управления обучением, программная платформа, предназначенная для организации, управления и сопровождения образовательного процесса.

No-code — подход к разработке программного обеспечения, позволяющий создавать цифровые продукты без необходимости написания программного кода.

MVP (Minimum Viable Product) — минимально жизнеспособный продукт, версия продукта с базовым функционалом, предназначенная для тестирования гипотезы и получения обратной связи от пользователей.

API (Application Programming Interface) — программный интерфейс приложения, набор методов и протоколов взаимодействия между программными компонентами.

BC (Bounded Context) — ограниченный контекст в методологии Domain-Driven Design; в работе обозначает автономный микросервис платформы Adapstory с собственной базой данных и доменной моделью (BC-01, BC-02, BC-10, BC-15, BC-16 и др.).

BFF (Backend For Frontend) — специализированный шлюз API, обслуживающий конкретный клиентский интерфейс.

BM25 — алгоритм лексического ранжирования документов на основе вероятностной модели релевантности.

C4 — модель визуализации архитектуры программных систем, включающая четыре уровня абстракции (Context, Container, Component, Code).

CI (Continuous Integration) — практика непрерывной интеграции изменений в общую кодовую базу с автоматической проверкой качества.

CPU (Central Processing Unit) — центральный процессор.

DTO (Data Transfer Object) — объект передачи данных между слоями приложения, не содержащий бизнес-логики.

GitOps — подход к управлению инфраструктурой Kubernetes, при котором git-репозиторий выступает источником истины, а изменения в кластер вносятся только через автоматическую синхронизацию.

GPT (Generative Pre-trained Transformer) — семейство больших языковых моделей компании OpenAI, основанных на архитектуре Transformer.

GPU (Graphics Processing Unit) — графический процессор, используемый в том числе для ускорения вычислений в задачах машинного обучения.

HNSW (Hierarchical Navigable Small World) — алгоритм приближённого поиска ближайших соседей в векторных пространствах.

HTTP (HyperText Transfer Protocol) — прикладной протокол передачи данных в сети.

ITS (Intelligent Tutoring Systems) — интеллектуальные обучающие системы, использующие методы искусственного интеллекта для адаптации образовательного контента.

JSON (JavaScript Object Notation) — текстовый формат обмена структурированными данными.

JTBD (Jobs To Be Done) — методология выявления реальных задач пользователя, лежащих в основе использования продукта.

JWT (JSON Web Token) — стандарт компактной передачи аутентифицированной информации в формате JSON.

LOC (Lines of Code) — строки кода; мера объёма исходного кода программы.

LSTM (Long Short-Term Memory) — архитектура рекуррентной нейронной сети с долгой краткосрочной памятью.

ML (Machine Learning) — машинное обучение, раздел искусственного интеллекта, изучающий методы построения моделей, обучаемых на данных.

MRR (Mean Reciprocal Rank) — средний обратный ранг; метрика качества ранжирования первого релевантного результата.

NDCG (Normalized Discounted Cumulative Gain) — нормализованный дисконтированный совокупный прирост; метрика качества ранжирования с учётом позиции релевантных результатов.

NLP (Natural Language Processing) — обработка естественного языка, раздел искусственного интеллекта, посвящённый алгоритмам анализа и генерации текста.

NPS (Net Promoter Score) — индекс потребительской лояльности.

OCR (Optical Character Recognition) — оптическое распознавание символов; технология преобразования изображений

текста в редактируемый текст.

PDB (Pod Disruption Budget) — бюджет допустимых одновременных нарушений работы подов в Kubernetes.

PDF (Portable Document Format) — формат электронных документов с фиксированной разметкой.

PII (Personally Identifiable Information) — персональные данные, позволяющие идентифицировать физическое лицо.

RAG (Retrieval-Augmented Generation) — архитектура генерации ответов большой языковой моделью с дополнением контекстом, извлечённым из внешней базы знаний.

RAGAS — набор автоматических метрик качества RAG-систем, включающий точность ответов, полноту извлечения и релевантность контекста.

REST (Representational State Transfer) — архитектурный стиль построения распределённых веб-сервисов.

RPS (Requests Per Second) — число запросов в секунду; мера нагрузки на сервис.

SaaS (Software as a Service) — модель распространения программного обеспечения как облачной услуги.

SBOM (Software Bill of Materials) — реестр программных компонентов и их зависимостей в составе артефакта.

SDK (Software Development Kit) — комплект средств разработки для интеграции с конкретной платформой или сервисом.

SLI (Service Level Indicator) — показатель фактического уровня обслуживания сервиса.

SLO (Service Level Objective) — целевое значение показателя уровня обслуживания.

SRS (Spaced Repetition System) — система интервального повторения, основанная на модели забывания.

TTL (Time To Live) — время жизни записи в кэше или хранилище, по истечении которого запись считается устаревшей.

TTS (Text-to-Speech) — синтез речи из текста.

UI (User Interface) — пользовательский интерфейс.

UUID (Universally Unique Identifier) — универсально уникальный идентификатор.

UX (User Experience) — пользовательский опыт.

VLM (Vision Language Model) — мультимодальная языковая модель, способная обрабатывать как текст, так и изображения.

WCAG (Web Content Accessibility Guidelines) — рекомендации по доступности веб-контента для пользователей с ограниченными возможностями.

БД — база данных.

ИИ — искусственный интеллект (см. AI).

СУБД — система управления базами данных.

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Адаптивное обучение — подход к организации образовательного процесса, при котором содержание, темп и траектория обучения изменяются в зависимости от индивидуальных особенностей, уровня подготовки и прогресса обучающегося.

Интеллектуальный образовательный помощник — программный инструмент на основе технологий искусственного интеллекта, предназначенный для поддержки обучающихся и преподавателей в процессе обучения, включая ответы на вопросы, рекомендации по обучению и автоматизацию проверки заданий.

Образовательная платформа — программно-технологическая система, обеспечивающая размещение учебных материалов, управление образовательным процессом, взаимодействие между участниками обучения и анализ образовательных данных.

Персонализация обучения — процесс адаптации образовательного контента, методов и инструментов обучения с учётом индивидуальных особенностей обучающихся.

Цифровая образовательная среда — совокупность цифровых технологий, платформ и сервисов, обеспечивающих организацию и реализацию образовательного процесса с использованием информационно-коммуникационных технологий.

AI-агент — программная система на основе технологий искусственного интеллекта, способная выполнять определённые задачи автономно или совместно с пользователем, анализируя данные и принимая решения в рамках заданной функциональности.

Адаптивное обучение — подход к организации образовательного процесса, при котором содержание, темп и траектория обучения изменяются в зависимости от индивидуальных особенностей, уровня подготовки и прогресса обучающегося.

Виджет — типизированный декларативный элемент пользовательского интерфейса в платформе Adapstory, описываемый JSON-структурой и обладающий собственными метаданными (категория, критичность, размер, набор допустимых действий, время жизни); виджеты собираются на экране в адаптивную сетку Widget Lake.

Галлюцинация большой языковой модели — феномен генерации правдоподобного, но фактически недостоверного содержимого языковой моделью при отсутствии или недостаточности релевантных знаний; в образовательном контексте представляет особую опасность, поскольку студент может сформировать устойчивое заблуждение на основе неверного ответа системы.

Интеллектуальный образовательный помощник — программный инструмент на основе технологий искусственного интеллекта, предназначенный для поддержки обучающихся и преподавателей в процессе обучения, включая ответы на вопросы, рекомендации по обучению и автоматизацию проверки заданий.

Когнитивный профиль студента — индивидуальный набор параметров, описывающий предпочтительный стиль

обучения конкретного учащегося: глубина объяснений, предпочтение абстрактных или конкретных примеров, темп взаимодействия, языковой регистр, чувствительность к обратной связи; формируется на основе явных настроек, поведенческой телеметрии и моделей коллаборативной фильтрации.

Образовательная платформа — программно-технологическая система, обеспечивающая размещение учебных материалов, управление образовательным процессом, взаимодействие между участниками обучения и анализ образовательных данных.

Персонализация обучения — процесс адаптации образовательного контента, методов и инструментов обучения с учётом индивидуальных особенностей обучающихся.

Плагин — расширение платформы, поставляющее дополнительные образовательные возможности (типы виджетов, сценарии тьюторинга, интеграции с внешними сервисами) в виде самостоятельного версионированного артефакта с декларируемым манифестом возможностей.

Промпт — текстовая инструкция, передаваемая большой языковой модели для управления её поведением; в архитектуре Smart Line промпты хранятся в реестре с семантической версией и криптографическим отпечатком, изменение шаблона требует выпуска новой версии.

Семантический поиск — извлечение фрагментов знаний из корпуса документов на основе близости их векторных представлений к векторному представлению запроса, в отличие от лексического поиска по точному совпадению термов.

Сократический диалог — педагогический метод, при котором наставник не сообщает готовый ответ, а ведёт обучающегося к самостоятельному выводу через последовательность наводящих вопросов; реализован в Smart Line через стратегию тьюторского агента *socratic*.

Стратегия тьюторского агента — формализованный режим педагогического воздействия, выбираемый агентом-планировщиком на каждом шаге сессии: *socratic* (наводящий вопрос), *hint* (краткая подсказка), *explain* (развёрнутое объяснение со ссылкой на источник), *verify* (проверочный вопрос после объяснения), *mastered* (терминальное состояние освоения).

Чанк — фрагмент текстового документа фиксированного размера (в работе — 400 слов с перекрытием 50 слов), полученный методом скользящего окна и индексируемый в векторной базе данных как отдельная единица поиска.

AI-тьютор — программная система на основе больших языковых моделей, реализующая функции наставника в учебном процессе: объяснение материала, формирование наводящих вопросов, оценку ответов студента, генерацию подсказок и адаптацию объяснения под уровень обучающегося.

GenAI-native — архитектурный принцип, согласно которому большая языковая модель рассматривается не как вспомогательный API, а как основной исполнительный компонент системы, и каждая сквозная функция описывается в терминах роли LLM, механизмов управляемости вывода, контроля знаний и измеримого образовательного эффекта.

Retrieval-Augmented Generation — методологический подход к работе с большими языковыми моделями, при

котором генерация ответа выполняется не из весов модели, а на основе динамически извлечённого контекста: запрос векторизуется, выполняется семантический поиск по корпусу документов, и найденные фрагменты передаются в языковую модель вместе с исходным вопросом; обеспечивает «заземление» модели на конкретном корпусе и контролируемость источников знаний.

Smart Line — единая точка входа студента ко всем AI-возможностям платформы Adapstory, реализованная как плавающая нижняя панель интерфейса с поддержкой текстового и голосового ввода и адаптивной сеткой виджетов Widget Lake; объединяет функции тьютора учебного содержания, помощника по организационным вопросам, навигации по каталогу, рефлексии прогресса и настройкам профиля.

ВВЕДЕНИЕ

Развитие цифровых технологий и распространение интернет-сервисов привели к существенной трансформации образовательной сферы. В последние годы наблюдается устойчивый рост интереса к онлайн-образованию и дистанционным образовательным технологиям. Согласно исследованиям российского рынка EdTech, объем рынка онлайн-образования в России стабильно увеличивается и сопровождается ростом числа пользователей образовательных платформ. В частности, аналитические отчеты отмечают, что цифровые образовательные сервисы становятся одним из ключевых инструментов повышения доступности и гибкости образования, позволяя обучающимся получать знания независимо от территориальных и временных ограничений [1].

В условиях активного развития цифровой образовательной среды возрастает роль систем управления обучением (Learning Management Systems, LMS). Такие системы позволяют организовывать учебный процесс, размещать образовательные материалы, отслеживать прогресс обучающихся и обеспечивать коммуникацию между преподавателями и студентами [2]. Вместе с тем традиционные LMS-платформы нередко ориентированы на стандартные образовательные сценарии и не обеспечивают достаточной персонализации обучения. Это ограничивает возможности преподавателей в адаптации образовательного процесса под индивидуальные особенности студентов.

Одной из ключевых проблем современного онлайн-образования является масштабирование образовательного процесса при сохранении высокого качества обучения. В условиях увеличения числа обучающихся преподаватели сталкиваются с существенным ростом нагрузки, связанной с проверкой заданий, подготовкой обратной связи и сопровождением студентов. Особенно остро данная проблема проявляется в онлайн-школах и образовательных проектах, где количество обучающихся может достигать нескольких сотен или тысяч человек. В результате снижается скорость обратной связи, ухудшается качество образовательного сопровождения и уменьшается вовлеченность студентов в процесс обучения.

Согласно исследованиям российского онлайн-образования, наличие регулярной обратной связи и взаимодействия с преподавателем является одним из ключевых факторов успешного обучения и удовлетворенности студентов образовательными программами [3]. Однако при значительном увеличении масштабов обучения обеспечение такого уровня взаимодействия становится затруднительным. В этой связи возрастает необходимость поиска новых технологических решений, позволяющих автоматизировать часть образовательных процессов без снижения качества обучения.

Одним из перспективных направлений развития образовательных технологий является применение методов искусственного интеллекта и технологий обработки естественного языка. В последние годы особое внимание привлекают большие языковые модели (Large Language Models), способные анализировать текстовую информацию, генерировать ответы на вопросы, проверять задания и формировать рекомендации по обучению [4]. Использование таких технологий открывает новые возможности для создания интеллектуальных образовательных систем, способных поддерживать студентов в процессе обучения и помогать преподавателям в организации образовательного процесса.

В российской научной и образовательной литературе активно рассматриваются вопросы цифровизации образования, развития онлайн-платформ и внедрения технологий искусственного интеллекта в образовательный процесс. Исследования посвящены анализу эффективности дистанционного обучения, особенностям функционирования LMS-систем, а также возможностям использования интеллектуальных систем поддержки обучения [5]. Однако вопросы интеграции больших языковых моделей в образовательные платформы и создания адаптивных образовательных сред с использованием AI-ассистентов остаются недостаточно разработанными.

Таким образом, актуальность темы исследования обусловлена необходимостью разработки новых цифровых образовательных инструментов, позволяющих эффективно масштабировать образовательный процесс, повышать уровень персонализации обучения и обеспечивать качественную обратную связь между

преподавателями и студентами при большом количестве обучающихся.

Объектом исследования является цифровая образовательная среда онлайн-обучения.

Предметом исследования являются методы и инструменты разработки образовательных платформ с использованием технологий искусственного интеллекта и больших языковых моделей для поддержки образовательного процесса.

Целью данной выпускной квалификационной работы является разработка бизнес-концепции образовательной платформы Adapstory, реализующей интеллектуального помощника в обучении на основе больших языковых моделей для автоматизации образовательных процессов и повышения эффективности онлайн-обучения.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) проанализировать современное состояние и тенденции развития рынка онлайн-образования и образовательных платформ в России;
- 2) рассмотреть особенности функционирования систем управления обучением (LMS) и их роль в цифровой образовательной среде;
- 3) выявить основные проблемы масштабирования образовательного процесса и организации обратной связи в онлайн-обучении;
- 4) исследовать возможности применения технологий искусственного интеллекта и больших языковых моделей в образовательных системах;
- 5) разработать концепцию образовательной платформы Adapstory;
- 6) описать механизм функционирования платформы и её основные функциональные возможности;
- 7) определить целевую аудиторию проекта и бизнес-модель платформы.

Гипотеза исследования заключается в предположении о том, что внедрение образовательной платформы, использующей большие языковые модели в качестве интеллектуального помощника преподавателя, позволит существенно повысить эффективность онлайн-обучения за счет автоматизации проверки заданий, персонализации образовательных траекторий и обеспечения оперативной обратной связи со студентами.

Научная новизна работы заключается в разработке концепции образовательной платформы нового поколения, в которой преподаватель и интеллектуальный AI-агент выступают равноправными участниками образовательного процесса. В рамках предложенной концепции предусматривается использование больших языковых моделей для:

- автоматизации проверки учебных заданий;
- анализа образовательных данных студентов;
- формирования персонализированных образовательных траекторий;
- поддержки студентов посредством интеллектуального ассистента.

Дополнительным элементом новизны является предложенная архитектура платформы, основанная на многоуровневой системе расширяемости, позволяющей пользователям создавать и модифицировать образовательные инструменты с различным уровнем технической подготовки — от визуального конструктора до AI-генерации плагинов.

Практическая значимость работы заключается в разработке концепции образовательной платформы Adapstory, ориентированной на использование в онлайн-школах, образовательных акселераторах и других организациях, реализующих программы дистанционного обучения.

Результаты исследования могут быть использованы для:

- разработки и внедрения интеллектуальных образовательных платформ;
- повышения эффективности организации онлайн-обучения;
- снижения нагрузки на преподавателей при работе с большими потоками студентов;
- повышения уровня персонализации образовательных программ.

Реализация предложенной платформы может способствовать повышению качества образовательных услуг и расширению возможностей цифровой образовательной среды.

Проект Adapstory представляет собой образовательную платформу нового поколения (AI LMS), предназначенную для создания персонализированных образовательных сред с использованием технологий искусственного интеллекта.

Основная идея платформы заключается в совместной работе преподавателя и интеллектуального AI-агента, которые выступают равноправными участниками образовательного процесса. Платформа предоставляет инструменты для создания образовательных курсов, анализа прогресса студентов, автоматической проверки заданий и формирования индивидуальных образовательных траекторий.

Архитектура платформы предусматривает четырёхуровневую систему расширяемости:

- визуальный конструктор образовательных курсов (no-code);
- подключаемые образовательные плагины;
- возможность разработки пользовательских модулей;
- генерацию инструментов с помощью искусственного интеллекта.

Основными пользователями платформы являются онлайн-школы и образовательные акселераторы, которым требуется гибкая и масштабируемая LMS-система без необходимости сложной разработки. Использование платформы позволяет преподавателям автоматизировать рутинные процессы, повысить эффективность взаимодействия со студентами и масштабировать образовательные программы.

1 АНАЛИЗ РЫНКА EDTECH И ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА В ОБРАЗОВАНИИ

В данном разделе исследуем мировой и российский рынок EDTECH.

1.1 Анализ российского рынка EdTech

В последние годы цифровая трансформация образования стала одним из ключевых факторов развития образовательной индустрии. Активное распространение интернет-технологий, мобильных устройств и облачных сервисов привело к формированию нового сегмента рынка — образовательных технологий (EdTech). Под EdTech понимается совокупность цифровых инструментов и платформ, направленных на повышение эффективности образовательного процесса, автоматизацию обучения и обеспечение доступа к образовательным ресурсам независимо от географического положения пользователя.

Рост EdTech-сектора особенно ускорился после пандемии COVID-19, когда многие образовательные учреждения и компании были вынуждены перейти на дистанционные форматы обучения. В результате цифровые образовательные платформы стали важным элементом образовательной инфраструктуры.

Одним из ключевых направлений развития EdTech является использование технологий искусственного интеллекта (AI). Такие технологии позволяют анализировать образовательные данные, автоматизировать проверку заданий, формировать адаптивные образовательные траектории и обеспечивать поддержку студентов в процессе обучения. В частности, развитие больших языковых моделей (Large Language Models) открыло новые возможности для создания интеллектуальных образовательных помощников и AI-тьюторов.

Современные AI-ассистенты способны выполнять широкий спектр задач в образовательной среде:

- отвечать на вопросы студентов в режиме реального времени;
- генерировать учебные материалы;
- проверять задания и давать обратную связь;
- анализировать прогресс обучающихся;
- формировать индивидуальные образовательные рекомендации.

Исследования показывают, что использование интеллектуальных систем обучения позволяет значительно повысить эффективность образовательного процесса. Например, исследования в области интеллектуальных обучающих систем демонстрируют рост успеваемости студентов и повышение скорости освоения учебного материала благодаря персонализированным рекомендациям и автоматической обратной связи [6].

Российский рынок образовательных технологий демонстрирует устойчивый рост в течение последних лет. По данным аналитического агентства Smart Ranking, совокупная выручка 100 крупнейших компаний EdTech-рынка России в третьем квартале 2025 года составила около 42 млрд рублей, что соответствует росту примерно на 12

Рост рынка обусловлен несколькими ключевыми факторами:

- цифровизация образовательной среды;
- рост спроса на профессиональное обучение и переквалификацию;
- развитие дистанционного образования;
- распространение технологий искусственного интеллекта.

Особенно заметным драйвером роста стали программы обучения, связанные с технологиями искусственного интеллекта и анализом данных. Образовательные платформы отмечают рост интереса к курсам по машинному обучению, аналитике данных и нейросетям [7].

В целом рынок EdTech в России можно разделить на несколько основных сегментов:

- 1) школьное онлайн-образование;
- 2) дополнительное профессиональное образование (ДПО);
- 3) корпоративное обучение;

- 4) языковое обучение;
- 5) платформенные решения и инфраструктурные сервисы.

Российский рынок образовательных технологий характеризуется высокой концентрацией крупных игроков, которые формируют значительную часть выручки отрасли.

Одним из крупнейших игроков является образовательный холдинг Skillbox. Платформа специализируется на обучении цифровым профессиям и предлагает курсы по программированию, дизайну, маркетингу и анализу данных. По данным отраслевых исследований, количество платных пользователей образовательных платформ холдинга приближается к миллиону, а спрос на курсы, связанные с искусственным интеллектом и машинным обучением, продолжает расти [8].

Skillbox активно интегрирует технологии искусственного интеллекта в образовательный процесс. Использование автоматизированных систем проверки заданий и анализа прогресса студентов позволяет масштабировать обучение и снижать нагрузку на преподавателей.

Крупным игроком на рынке онлайн-образования является образовательная компания Skyeng. Компания специализируется на онлайн-обучении английскому языку и использует собственную образовательную платформу с элементами адаптивного обучения.

Skyeng активно внедряет технологии искусственного интеллекта для анализа учебных данных и персонализации образовательного процесса. Использование автоматизированных рекомендаций и интеллектуальных систем подбора заданий позволяет улучшать результаты обучения студентов.

Платформа Яндекс Практикум является одним из крупнейших проектов в сфере онлайн-обучения цифровым профессиям. Основными направлениями обучения являются программирование, анализ данных и цифровой маркетинг.

Платформа активно применяет инструменты автоматизированной проверки заданий и аналитики образовательных данных, а также внедряет элементы искусственного интеллекта для персонализации образовательных траекторий.

Платформа Учи.ру является одним из крупнейших образовательных сервисов для школьников. Она предоставляет интерактивные задания по школьным предметам и активно использует алгоритмы адаптивного обучения.

Исследования показывают, что использование интеллектуальных систем подбора заданий позволяет сократить время подготовки преподавателей примерно на треть, что подтверждает эффективность применения технологий искусственного интеллекта в образовательной среде [9].

GetCourse представляет собой инфраструктурную платформу для создания онлайн-школ и образовательных проектов. Платформа предоставляет инструменты для управления курсами, обработки платежей и взаимодействия с учениками.

Данный тип платформ является важным элементом экосистемы EdTech, поскольку позволяет преподавателям и образовательным компаниям запускать собственные образовательные продукты без необходимости разработки сложной IT-инфраструктуры.

Современные образовательные платформы в России постепенно внедряют технологии искусственного интеллекта. Однако в большинстве случаев AI-решения пока используются для автоматизации отдельных задач, а не для полной трансформации образовательного процесса.

Наиболее распространённые сценарии применения AI включают:

- автоматическую проверку домашних заданий;
- чат-боты поддержки студентов;
- генерацию образовательных материалов;
- анализ прогресса обучающихся.

По мнению экспертов рынка EdTech, искусственный интеллект в образовании пока не заменяет преподавателей, но позволяет автоматизировать рутинные задачи и повысить эффективность образовательного процесса [10].

1.2 Анализ мирового рынка EdTech

Мировой рынок образовательных технологий демонстрирует значительно более высокие темпы роста по сравнению с российским рынком.

По данным аналитических исследований, мировой рынок искусственного интеллекта в образовании оценивался примерно в 5,88 млрд долларов в 2024 году и может достичь 32 млрд долларов к 2030 году, демонстрируя среднегодовой темп роста более 30 процентов [11].

В свою очередь, рынок AI-тьюторов также активно развивается. Его объём оценивается примерно в 2 млрд долларов в 2025 году и может превысить 10 млрд долларов к 2032 году [12].

Основными факторами роста мирового EdTech-рынка являются:

- распространение дистанционного обучения;
- рост спроса на непрерывное образование;
- развитие технологий искусственного интеллекта;
- цифровизация образовательных учреждений.

Coursera является одной из крупнейших образовательных платформ в мире. Платформа сотрудничает с ведущими университетами и компаниями и предоставляет онлайн-курсы, профессиональные сертификаты и образовательные программы.

Компания активно внедряет технологии искусственного интеллекта для персонализации обучения и анализа образовательных данных. Кроме того, Coursera активно развивает корпоративное обучение и программы повышения квалификации.

Duolingo является одним из самых популярных приложений для изучения иностранных языков. Платформа использует элементы геймификации и адаптивного обучения для повышения вовлечённости пользователей.

Компания активно внедряет генеративный искусственный интеллект для создания образовательного контента и масштабирования образовательных программ. Благодаря использованию AI платформа смогла значительно ускорить разработку новых курсов и расширить количество доступных языков [13].

Khan Academy является некоммерческой образовательной платформой, предоставляющей бесплатные курсы по математике, программированию и другим дисциплинам.

Платформа активно внедряет AI-ассистентов, например интеллектуального помощника Khanmigo, который помогает студентам решать задачи, объясняет учебный материал и поддерживает обучающихся в процессе обучения.

Pearson — одна из крупнейших образовательных компаний в мире, специализирующаяся на разработке учебных материалов и цифровых образовательных решений.

Компания активно инвестирует в технологии искусственного интеллекта и адаптивного обучения, создавая интеллектуальные образовательные платформы и системы анализа образовательных данных.

Brainly представляет собой образовательную платформу, основанную на принципах коллективного обучения. Пользователи платформы задают вопросы и получают ответы от других участников образовательного сообщества.

В последние годы Brainly активно внедряет AI-ассистентов для автоматической генерации ответов и модерации контента.

Проведённый анализ показывает, что рынок EdTech продолжает активно развиваться как в России, так и на мировом уровне.

Российский рынок характеризуется постепенным ростом и формированием крупных образовательных платформ, однако внедрение технологий искусственного интеллекта находится на стадии активного развития. Мировой рынок EdTech демонстрирует более высокие темпы роста, что связано с более высоким уровнем инвестиций, развитием технологической инфраструктуры и активным внедрением технологий искусственного интеллекта.

Одним из ключевых направлений развития образовательных технологий является создание интеллектуальных образовательных платформ и AI-тьюторов, способных обеспечить персонализацию обучения и масштабирование образовательного процесса.

Таким образом, разработка образовательных платформ с использованием больших языковых моделей и интеллектуальных ассистентов является перспективным направлением развития EdTech-индустрии и может способствовать повышению эффективности онлайн-обучения.

1.3 Анализ проблемы организации онлайн-обучения и потребностей преподавателей

Развитие онлайн-образования и цифровых образовательных платформ привело к значительному расширению возможностей обучения, однако одновременно выявило ряд системных проблем, связанных с организацией образовательного процесса. Несмотря на активное внедрение технологий, включая системы управления обучением (LMS) и инструменты автоматизации, значительная часть нагрузки по-прежнему лежит на преподавателях.

Для выявления ключевых проблем и особенностей организации онлайн-обучения были проведены качественные интервью с преподавателями, работающими в различных форматах: индивидуальное обучение взрослых, работа в онлайн-школе с детьми, а также проведение массовых образовательных программ и марафонов. Анализ интервью позволил детально рассмотреть процесс работы преподавателя и выделить основные этапы образовательного цикла, а также возникающие на каждом этапе сложности.

Анализ этапов образовательного процесса

В ходе исследования было выявлено, что работа преподавателя представляет собой последовательность взаимосвязанных этапов, каждый из которых сопровождается определёнными целями, задачами и проблемами. На начальном этапе преподаватель определяет формат обучения, целевую аудиторию, цели курса и уровень сопровождения. Данный этап является критически важным, поскольку от него зависит структура всего образовательного процесса.

Однако результаты интервью показывают, что преподаватели сталкиваются с рядом сложностей:

- отсутствие точного понимания уровня подготовки студентов;
- трудности в прогнозировании нагрузки;
- необходимость адаптации формата обучения под разные аудитории;
- зависимость от внешних условий (например, требований онлайн-школы или продюсера курса).

Таким образом, уже на начальном этапе возникает неопределённость, которая влияет на дальнейшую организацию обучения.

Следующий этап связан с подготовкой всех необходимых ресурсов для проведения обучения: материалов, заданий, ссылок, доступов и информации о студентах.

Анализ интервью показал, что одной из ключевых проблем является фрагментированность информации:

- материалы распределены по различным сервисам (Google Drive, мессенджеры, LMS, доски);
- отсутствует единое пространство хранения данных;
- возникают сложности с доступами и актуальностью информации;
- коммуникация с участниками осуществляется через несколько каналов.

Это приводит к увеличению временных затрат и снижению управляемости процесса.

Подготовка материалов и заданий является одним из наиболее трудоёмких этапов. Преподаватель должен не только структурировать материалы, но и разработать практические задания, адаптированные под уровень аудитории.

По результатам интервью, данный этап характеризуется следующими проблемами:

- высокая когнитивная нагрузка при разработке заданий;
- сложность создания практики, одновременно полезной и понятной;
- необходимость адаптации контента под разные уровни подготовки;
- отсутствие инструментов автоматизации генерации заданий.

Особенно остро данная проблема проявляется в случаях, когда преподаватель работает вне стандартных учебных программ и вынужден самостоятельно разрабатывать контент.

Перед началом обучения преподаватель должен убедиться, что все материалы, ссылки и доступы работают

корректно.

Основные сложности данного этапа:

- ошибки в ссылках и доступах;
- путаница между версиями материалов (черновики и финальные версии);
- риск несогласованности элементов курса;
- отсутствие автоматических инструментов проверки.

Даже незначительные ошибки на этом этапе могут привести к сбоям в процессе обучения.

Непосредственное проведение занятий включает объяснение материала, взаимодействие со студентами и выдачу заданий.

Интересно, что данный этап не воспринимается преподавателями как основной источник проблем. Однако при увеличении масштаба обучения возникают новые сложности:

- большое количество повторяющихся вопросов;
- необходимость постоянной коммуникации с участниками;
- эмоциональная нагрузка;
- управление вниманием аудитории.

Особенно выражены данные проблемы при работе с массовыми форматами обучения.

Контроль выполнения заданий и анализа прогресса студентов является важной частью образовательного процесса.

В ходе интервью были выявлены следующие проблемы:

- отсутствие единой системы мониторинга;
- сложность отслеживания дедлайнов и выполнения заданий;
- невозможность быстро выявить проблемные зоны;
- отсутствие аналитики по типичным ошибкам студентов.

В результате преподаватели часто принимают решения на основе интуиции, а не данных.

В процессе обучения преподавателю необходимо регулярно вносить изменения в структуру курса, задания и формат взаимодействия.

Основные сложности:

- необходимость ручной адаптации материалов;
- зависимость от технических ограничений платформ;
- отсутствие рекомендаций по улучшению процесса;
- высокая нагрузка при работе с разнородной аудиторией.

Данный этап требует значительных временных и когнитивных ресурсов.

Финальный этап включает подведение итогов, оценку результатов и подготовку отчётности.

Основные проблемы:

- сложность организации итоговой проверки;
- высокая трудоёмкость формирования обратной связи;
- необходимость подготовки отчётов;
- отсутствие автоматизации аналитики.

Кроме того, преподаватели отмечают сложности с фиксацией результатов обучения и подготовкой следующего цикла.

Анализ всех этапов позволяет выделить ключевые системные проблемы:

- 1) Фрагментация инструментов и данных. Образовательный процесс распределён между множеством сервисов, что снижает его управляемость.
- 2) Высокая операционная нагрузка. Значительная часть времени преподавателя уходит на организационные задачи, а не на обучение.
- 3) Недостаточная автоматизация. Существующие платформы не покрывают полный цикл образовательного процесса.
- 4) Отсутствие персонализации на уровне процесса. Несмотря на наличие адаптивных технологий, преподаватели вынуждены вручную адаптировать обучение.

- 5) Коммуникационные разрывы. Информация часто теряется или передаётся с задержками.
- 6) Высокая когнитивная и эмоциональная нагрузка. Преподаватели испытывают перегрузку из-за необходимости удерживать множество процессов одновременно.

Для более глубокого понимания потребностей пользователей был применён подход Jobs To Be Done (JTBD), позволяющий выявить реальные задачи, которые пользователь пытается решить.

В ходе исследования были выделены три ключевых сценария:

- 1) Самостоятельный преподаватель. Стремится объединить все процессы обучения в единую систему и снизить хаос в организации.
- 2) Преподаватель в онлайн-школе. Сталкивается с проблемами коммуникации и зависимостью от посредников.
- 3) Эксперт массовых курсов. Испытывает перегрузку из-за масштаба аудитории и необходимости обрабатывать большое количество взаимодействий.

Обобщённая формулировка задачи пользователя:

Когда преподаватель сопровождает онлайн-обучение в условиях ограниченного времени, разрозненных инструментов и нестабильной коммуникации, он стремится видеть образовательный процесс как единое управляемое целое и получать поддержку в рутинных задачах, чтобы сохранить контроль и сосредоточиться на качестве обучения.

Проведённый анализ показал, что ключевые сложности преподавателей связаны не столько с самим процессом преподавания, сколько с организацией и координацией образовательного процесса.

Наиболее критичными являются следующие аспекты:

- подготовка материалов и заданий;
- организация и структурирование информации;
- отслеживание прогресса студентов;
- коммуникация с участниками обучения.

Таким образом, основная потребность пользователей заключается в снижении операционной нагрузки и повышении управляемости образовательного процесса.

Данные выводы подтверждают необходимость разработки образовательной платформы нового поколения, в которой:

- все этапы обучения объединены в единую систему;
- автоматизированы рутинные задачи;
- обеспечивается поддержка преподавателя с помощью интеллектуальных инструментов;
- реализована персонализация обучения на уровне процесса, а не только контента.

Такая концепция лежит в основе разрабатываемого проекта Adapstory, ориентированного на интеграцию AI-ассистента в образовательную среду и поддержку преподавателя на всех этапах образовательного цикла.

1.4 Обзор технологий и решений в области AI-тьюторов и интеллектуальных образовательных систем

Развитие цифровых образовательных технологий и рост объёма онлайн-обучения обусловили необходимость внедрения интеллектуальных решений, направленных на автоматизацию образовательного процесса и повышение его эффективности. Одним из наиболее перспективных направлений является использование технологий искусственного интеллекта, в частности больших языковых моделей, для создания AI-тьюторов и интеллектуальных образовательных систем.

Современные образовательные платформы активно используют технологии искусственного интеллекта для анализа данных, персонализации обучения и автоматизации взаимодействия с пользователями.

Ключевыми технологическими направлениями являются:

- машинное обучение (Machine Learning);
- обработка естественного языка (Natural Language Processing, NLP);
- большие языковые модели (LLM);

- рекомендательные системы;
- анализ образовательных данных (Learning Analytics).

По данным исследований, внедрение технологий искусственного интеллекта позволяет повысить эффективность обучения за счёт персонализации образовательного процесса и автоматизации обратной связи [14].

Особую роль играют большие языковые модели, которые способны обрабатывать текстовую информацию, генерировать ответы и выполнять задачи, связанные с обучением, включая объяснение материала и проверку заданий.

Одним из ключевых направлений развития EdTech являются интеллектуальные обучающие системы (Intelligent Tutoring Systems, ITS).

ITS представляют собой программные системы, использующие методы искусственного интеллекта для моделирования процесса обучения и адаптации образовательного контента под индивидуальные особенности обучающегося.

Согласно исследованиям, интеллектуальные обучающие системы могут обеспечивать эффективность обучения, сопоставимую с индивидуальным обучением с преподавателем [14].

Основные функции ITS включают:

- диагностику уровня знаний обучающегося;
- адаптацию образовательного контента;
- генерацию заданий;
- автоматическую проверку ответов;
- предоставление обратной связи в режиме реального времени.

Однако традиционные ITS имеют ряд ограничений, включая сложность разработки и ограниченность сценариев взаимодействия.

С развитием технологий обработки естественного языка появились новые возможности для создания более гибких и универсальных образовательных систем.

Большие языковые модели (LLM) позволяют реализовать функции AI-тьютора, способного:

- объяснять учебный материал;
- отвечать на вопросы студентов;
- генерировать примеры и задания;
- анализировать ошибки;
- адаптировать объяснение под уровень обучающегося.

По данным исследований, использование LLM в образовании позволяет значительно повысить вовлечённость студентов и улучшить качество обратной связи [15].

В отличие от традиционных ITS, LLM обладают следующими преимуществами:

- универсальность применения;
- возможность обработки неструктурированных данных;
- гибкость в генерации контента;
- способность к диалоговому взаимодействию.

В последние годы на рынке появились реальные решения, реализующие концепцию AI-тьютора.

AI-ассистент Khanmigo используется в образовательной платформе Khan Academy и помогает студентам решать задачи, объясняет материал и направляет процесс обучения.

Особенностью данного решения является использование диалогового подхода, при котором AI не просто даёт ответ, а помогает студенту прийти к решению самостоятельно.

Платформа Duolingo внедрила AI-ассистента, который помогает пользователям изучать языки, объясняет ошибки и моделирует диалоги.

Использование генеративного AI позволило значительно расширить возможности платформы и повысить уровень персонализации обучения [16].

Squirrel AI является одной из наиболее развитых систем адаптивного обучения, использующей технологии искусственного интеллекта для построения индивидуальных образовательных траекторий.

Система анализирует знания студентов и предлагает персонализированные задания, направленные на устранение пробелов в знаниях.

Большие языковые модели, такие как ChatGPT, активно используются в образовательной среде в качестве универсальных AI-ассистентов.

Они позволяют:

- генерировать учебные материалы;
- объяснять сложные темы;
- помогать в выполнении заданий;
- поддерживать диалог с обучающимся.

Однако их использование требует контроля со стороны преподавателя, так как возможны ошибки и неточности в ответах.

Несмотря на активное развитие AI-тьюторов, существующие решения имеют ряд ограничений:

- 1) Фрагментарность функционала. Большинство решений решает отдельные задачи (например, ответы на вопросы), но не покрывает весь образовательный процесс.
- 2) Отсутствие интеграции в LMS. AI-ассистенты часто существуют отдельно от систем управления обучением.
- 3) Недостаточная адаптация под преподавателя. Основной фокус делается на студента, а не на поддержку преподавателя.
- 4) Ограниченная персонализация процесса. Персонализация чаще касается контента, а не всей образовательной траектории.
- 5) Отсутствие системной аналитики. Многие решения не предоставляют комплексной картины образовательного процесса.

Анализ текущих технологий позволяет выделить ключевые направления развития AI-тьюторов: интеграция AI в образовательные платформы, создание систем полного цикла (end-to-end), развитие адаптивных образовательных траекторий; автоматизация коммуникации, снижение нагрузки на преподавателей.

По прогнозам аналитиков, рынок AI в образовании будет активно расти в ближайшие годы, а технологии AI-тьюторов станут стандартом для образовательных платформ [17].

Проведённый анализ показывает, что современные технологии искусственного интеллекта обладают значительным потенциалом для трансформации образовательного процесса.

В частности:

- интеллектуальные обучающие системы позволяют персонализировать обучение;
- большие языковые модели обеспечивают гибкость и масштабируемость образовательных решений;
- AI-тьюторы способны автоматизировать значительную часть образовательных процессов.

Однако существующие решения не обеспечивают комплексного подхода к организации обучения и не покрывают весь цикл работы преподавателя.

Таким образом, существует необходимость разработки образовательной платформы нового поколения, которая:

- объединяет все этапы образовательного процесса;
- интегрирует AI-ассистента в работу преподавателя;
- обеспечивает персонализацию обучения;
- снижает операционную нагрузку.

Данные требования реализуются в рамках проекта Adapstory, который направлен на создание AI-ориентированной образовательной платформы с поддержкой преподавателя на всех этапах обучения.

В рамках первой главы был проведён комплексный анализ рынка образовательных технологий, существующих решений в области онлайн-обучения, а также исследованы проблемы организации образовательного процесса и современные технологические подходы к их решению.

Анализ российского и мирового рынков EdTech показал устойчивый рост отрасли и активное внедрение цифровых технологий в образовательный процесс. При этом ключевыми тенденциями являются развитие персонализированного обучения, внедрение адаптивных образовательных систем и использование технологий искусственного интеллекта. Среди ведущих игроков рынка выявлены как глобальные платформы (Coursera,

Udemy, Khan Academy, Duolingo), так и российские образовательные экосистемы (Skillbox, Нетология, Яндекс Практикум), предлагающие различные подходы к организации обучения.

Сравнительный анализ платформ показал, что несмотря на разнообразие функциональных возможностей, большинство решений ориентировано преимущественно на доставку контента и управление курсами, в то время как поддержка преподавателя и автоматизация полного образовательного цикла реализованы недостаточно полно.

Проведённый анализ проблемы организации онлайн-обучения, основанный на интервью с преподавателями и применении подхода Jobs To Be Done (JTBD), позволил выявить ключевые трудности, возникающие на всех этапах образовательного процесса. К ним относятся высокая операционная и когнитивная нагрузка на преподавателя, фрагментация используемых инструментов, недостаточная автоматизация рутинных задач, сложности в отслеживании прогресса обучающихся и ограниченные возможности персонализации обучения. Установлено, что основная потребность пользователей заключается в создании единой управляемой образовательной среды, обеспечивающей интеграцию всех этапов обучения и снижение нагрузки на преподавателя за счёт автоматизации и интеллектуальной поддержки.

Обзор современных технологий показал, что значительный потенциал для решения выявленных проблем имеют технологии искусственного интеллекта, в частности большие языковые модели и интеллектуальные обучающие системы. Они позволяют реализовать функции AI-тьютора, включая генерацию контента, автоматическую проверку заданий, адаптацию образовательных траекторий и поддержку диалогового взаимодействия с обучающимися.

В то же время анализ существующих AI-решений выявил их ограниченность: они, как правило, реализуют отдельные функции и не обеспечивают комплексной поддержки преподавателя в рамках единой системы.

Таким образом, результаты проведённого исследования подтверждают актуальность разработки образовательной платформы нового поколения, ориентированной на интеграцию AI-ассистента в образовательный процесс и автоматизацию ключевых этапов работы преподавателя. Данная концепция реализуется в рамках проекта Adapstory, который направлен на создание единой AI LMS платформы, обеспечивающей персонализацию обучения, повышение его эффективности и масштабируемость без пропорционального увеличения нагрузки на преподавателя.

2 РАЗРАБОТКА И ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ MVP AI-ТЮТОРА

2.1 Дизайн эксперимента по разработке и оценке AI-тьютора

В рамках данной работы проводится разработка и экспериментальная оценка минимально жизнеспособного продукта (MVP) AI-тьютора, реализованного на основе технологий больших языковых моделей, архитектуры Retrieval-Augmented Generation (RAG) и агентного подхода.

Данный раздел посвящён формированию дизайна эксперимента, включающего постановку гипотез, описание сравниваемых архитектур, характеристику используемых данных, формализацию метрик оценки и определение методики проведения эксперимента.

Целью эксперимента является оценка эффективности различных архитектур AI-тьютора с точки зрения качества генерации образовательных ответов и их педагогической ценности.

Основная гипотеза исследования формулируется следующим образом: интеграция архитектуры Retrieval-Augmented Generation с агентным подходом позволяет повысить качество ответов AI-тьютора по сравнению с использованием только больших языковых моделей или RAG без агентного слоя.

Для формализации исследования вводятся статистические гипотезы:

- нулевая гипотеза (H0): использование различных архитектур (LLM, RAG, RAG + агент) не приводит к статистически значимому различию в качестве ответов;
- альтернативная гипотеза (H1): архитектура RAG + AI-агент обеспечивает более высокое качество ответов по сравнению с другими подходами.

Эксперимент построен по принципу сравнительного анализа трёх архитектурных подходов к реализации AI-тьютора:

генерация ответа с использованием только LLM; генерация ответа с использованием архитектуры RAG; генерация ответа с использованием архитектуры RAG с агентным управлением.

Общая схема эксперимента представлена на рисунке 2.1.

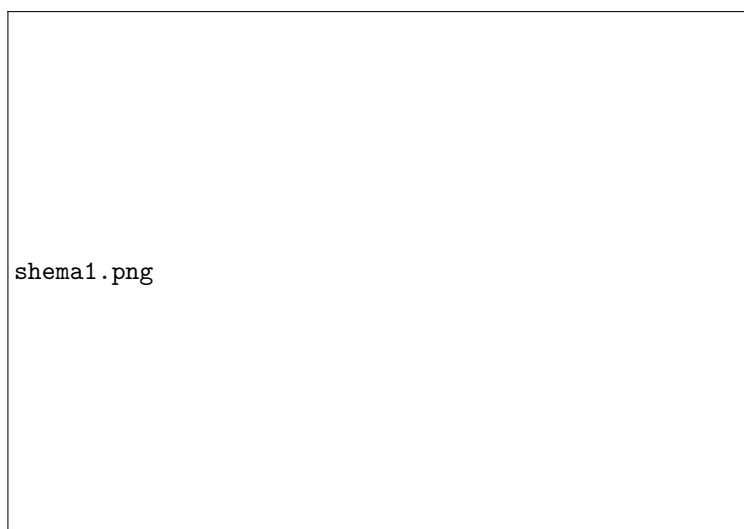


Рисунок 2.1 — Схема экспериментов

Таким образом, каждый пользовательский запрос обрабатывается тремя различными системами, после чего проводится их сравнительный анализ.

Первый сценарий представляет собой базовую модель, в которой ответ генерируется исключительно на основе параметров большой языковой модели без использования внешних данных.

Формально данный процесс можно описать как:

$$y = f(x), \quad (2.1)$$

где x — входной запрос пользователя, f — языковая модель, y — сгенерированный ответ.

Данный подход используется в качестве контрольной группы, позволяющей оценить вклад дополнительных механизмов (RAG и агентов).

Основные ограничения:

- отсутствие привязки к учебным материалам;
- возможность генерации недостоверной информации;
- отсутствие контроля источников знаний.

Во втором сценарии используется архитектура Retrieval-Augmented Generation, позволяющая дополнить генерацию ответов внешними знаниями.

Формально процесс можно представить как:

$$y = f(x, C(x)), \quad (2.2)$$

где $C(x)$ — контекст, полученный в результате поиска по базе знаний.

Контекст формируется на основе функции retrieval:

$$C(x) = Top - k(sim(E(x), E(D))), \quad (2.3)$$

где $E(x)$ — embedding запроса, $E(D)$ — embeddings документов, sim — функция сходства (например, косинусная близость), k — количество извлекаемых документов.

Третий сценарий расширяет архитектуру RAG за счёт добавления агентного слоя, который управляет процессом генерации ответа.

Формально:

$$y = f(x, C(x), S(x)), \quad (2.4)$$

где $S(x)$ — стратегия ответа, определяемая агентом.

Агент выполняет следующие функции:

- классификация запроса;
- выбор стратегии ответа;
- адаптация уровня объяснения;
- управление структурой ответа.

Таким образом, агентный слой позволяет перейти от простого ответа к педагогически ориентированному взаимодействию. Для проведения эксперимента используется база знаний, сформированная на основе образовательных материалов.

В набор данных входят:

- лекционные материалы;
- конспекты;
- практические задания;
- методические рекомендации;
- примеры решений.

Процесс подготовки данных включает несколько этапов: Документы → Очистка → Чанкинг → Embedding → Индексация

Удаляются: служебные символы, дублирующиеся фрагменты, нерелевантные данные. Тексты разбиваются на фрагменты фиксированного размера.

Параметры:

- размер чанка: 300–800 токенов;
- перекрытие: 10–20

Выбор размера чанка влияет на качество поиска: слишком крупные чанки снижают точность, слишком мелкие

— ухудшают контекст. Каждый фрагмент преобразуется в векторное представление.

Для оценки работы системы используются типовые образовательные сценарии.

Таблица 2.1 — Название таблицы

Тип сценария	Пример запроса	Цель
Теоретический вопрос	Объясни понятие градиентного спуска	Проверка объяснений
Непонимание	Я не понял тему	Проверка адаптации
Генерация примера	Приведи пример	Проверка генерации

Для оценки качества ответов используются следующие метрики.

Таблица 2.2 — Название таблицы

Метрика	Описание
Релевантность	Соответствие вопросу
Точность	Отсутствие ошибок
Полнота	Степень раскрытия темы
Объяснимость	Понятность объяснения

В рамках эксперимента используется стандартная архитектура RAG.

Вопрос → Embedding → Поиск → Контекст → LLM → Ответ

Процесс поиска основан на косинусной близости.

Агент реализует дополнительный уровень логики.

Вопрос → Агент → Классификация → RAG → LLM → Ответ

Функции агента: определение типа запроса, выбор стратегии, объяснение, пример, уточнение, адаптация ответа.

В рамках данного исследования архитектура AI-тьютора расширяется за счёт внедрения агентного слоя, обеспечивающего интеллектуальное управление процессом генерации ответа. В отличие от традиционных RAG-систем, где обработка запроса осуществляется линейно, агентный подход позволяет реализовать адаптивную и многошаговую логику взаимодействия.

Ответ формируется LLM с учётом: контекста, стратегии, инструкций агента.

По сравнению с классическим RAG, агент обеспечивает:

- 1) Адаптивность. Ответ подстраивается под пользователя.
- 2) Контекстно-зависимое поведение. Разные типы вопросов → разные стратегии
- 3) Педагогическую направленность. Ответ не просто информирует, а обучает
- 4) Многошаговую обработку. Возможность сложных сценариев.

Несмотря на преимущества, подход имеет ограничения:

- увеличение вычислительной сложности;
- зависимость от качества классификации;
- необходимость настройки промптов;
- риск ошибок в выборе стратегии.

Внедрение агентного слоя позволяет существенно расширить функциональность AI-тьютора, обеспечивая переход от генерации текстовых ответов к интеллектуальному сопровождению обучения. Агентный подход выступает ключевым элементом адаптивности системы и обеспечивает более высокий уровень педагогической эффективности по сравнению с традиционными архитектурами.

В результате разработки дизайна эксперимента были определены:

- три сравниваемых архитектуры (LLM, RAG, RAG + агент);
- набор образовательных сценариев;
- метрики оценки качества;
- методика обработки данных.

Сформированная экспериментальная схема позволяет объективно оценить влияние архитектурных решений на

качество работы AI-тьютора и служит основой для проведения практического исследования, представленного в последующих разделах.

2.2 Обзор используемых технологий и инструментов

Выбор технологического стека для разработки ИИ-тьютора определялся совокупностью функциональных, педагогических и операционных требований, сформулированных на этапе анализа предметной области. Центральным ограничением системы является требование отвечать на вопросы студентов исключительно на основе верифицированных учебных материалов компании Industrix, полностью исключая привлечение общих знаний языковой модели. Выполнение этого требования невозможно при использовании чистого генеративного подхода, при котором языковая модель опирается только на свои веса: такая система неизбежно генерирует ответы, выходящие за пределы учебного корпуса, и создаёт иллюзию наличия информации там, где её нет. Это явление, известное в литературе как «галлюцинация» языковой модели, представляет особую опасность в образовательном контексте: студент, получив правдоподобно сформулированный, но фактически неверный ответ, может сформировать устойчивое заблуждение, которое впоследствии будет трудно скорректировать.

Помимо требования ограниченности корпусом, система должна реализовывать педагогически обоснованный диалог, а не просто выдавать готовые ответы. Согласно принципам конструктивизма и сократического метода, знание усваивается значительно эффективнее, когда студент приходит к выводу самостоятельно — через управляемую последовательность вопросов и подсказок, — нежели при пассивном получении готового объяснения. Это педагогическое требование предопределяет необходимость многоступенчатой диалоговой архитектуры с анализом ответов студента и адаптивным выбором следующего педагогического шага.

Совокупность описанных ограничений предопределила выбор архитектурного паттерна Retrieval-Augmented Generation (RAG) как основы системы. В этом паттерне языковая модель получает знания не из весов нейросети, а из динамически извлекаемого контекста: на каждый входящий вопрос система сначала выполняет семантический поиск по корпусу документов, формирует контекст из найденных фрагментов и только затем передаёт их вместе с вопросом в языковую модель. Такой подход принципиально ограничивает модель рамками предоставленного контекста и позволяет явно инструктировать её: при отсутствии информации в контексте следует сообщить об этом, а не генерировать ответ из общих знаний. Паттерн RAG был предложен в работе и впоследствии получил широкое распространение в корпоративных приложениях именно благодаря свойству «заземления» модели на конкретном документальном корпусе.

Помимо архитектурного требования, при выборе конкретных инструментов учитывались: зрелость технологии и наличие активного сообщества, открытость к интеграции через стандартные API, возможность локального развёртывания компонентов, обрабатывающих потенциально конфиденциальные учебные материалы, а также минимизация стоимости инференса при сохранении качества, достаточного для педагогических задач.

В качестве основной языковой модели была выбрана **GPT-4o-mini** компании OpenAI. Данная модель представляет собой оптимизированную по стоимости версию семейства GPT-4o, обеспечивающую сопоставимое с предшественниками качество следования инструкциям при существенно меньшей стоимости токенов. По результатам оценки на стандартных бенчмарках, опубликованных OpenAI, GPT-4o-mini превосходит GPT-3.5-turbo на задачах понимания текста и следования сложным инструкциям. Для педагогических задач проекта — формирования наводящих вопросов, интерпретации ответов студентов и выбора стратегии обучения — возможностей модели оказывается достаточно: она корректно понимает многоуровневые инструкции и способна поддерживать контекст педагогического диалога. При этом задержка генерации ответа длиной 200–400 токенов составляет 2–4 секунды, что приемлемо для интерактивного учебного взаимодействия. Температура генерации устанавливается равной 0,2–0,3 в зависимости от типа агента: низкое значение обеспечивает детерминированность и точность следования инструкциям, а незначительное отклонение от нуля сохраняет вариативность формулировок, необходимую для естественности диалога.

Помимо GPT-4o-mini, при проектировании рассматривались альтернативные языковые модели: Claude

3.5 Haiku (Anthropic), Llama 3.1 8B (Meta) и Mistral 7B. Claude 3.5 Haiku демонстрирует сопоставимые характеристики с GPT-4o-mini, однако в период разработки API Anthropic был менее зрелым с точки зрения поддержки языка программирования Python. Llama 3.1 8B и Mistral 7B были рассмотрены как варианты локального развёртывания (через Ollama) без затрат на API, однако их качество следования сложным педагогическим инструкциям на русском языке оказалось недостаточным в ходе предварительных экспериментов. Таким образом, GPT-4o-mini был признан оптимальным решением по совокупности критериев.

Для построения векторных представлений текстов используется модель **text-embedding-3-large** с размерностью вектора 3 072 измерения. Данная модель является актуальным флагманским эмбедедером OpenAI и превосходит предшествующие поколения — text-embedding-ada-002 и text-embedding-3-small — по качеству семантического поиска на мультязычных корпусах, включая русскоязычные тексты. Использование высокоразмерного пространства обеспечивает более тонкое разделение семантически близких, но содержательно различных понятий — что особенно важно для предметной области предпринимательства, где термины нередко многозначны и контекстуально зависимы. Например, понятия «провал стартапа», «проблемы стартапа» и «риски предпринимательства» семантически близки, но различаются по смысловому акценту, и высокоразмерное пространство позволяет корректно ранжировать их при поиске. К полученным векторам применяется L2-нормализация: деление вектора на его евклидову норму обеспечивает корректную интерпретацию косинусного сходства в векторной базе данных.

Для хранения и поиска векторных представлений выбрана СУБД **Qdrant** [18] — высокопроизводительное векторное хранилище с открытым исходным кодом, написанное на языке Rust. Qdrant реализует алгоритм HNSW (Hierarchical Navigable Small World) для приближённого поиска ближайших соседей, обеспечивая сублинейную сложность запроса при высоком значении отзывчивости. Ключевыми преимуществами Qdrant в контексте проекта являются: встроенная поддержка фильтрации по полям полезной нагрузки (payload filtering), позволяющая реализовать мультиарендность без дублирования коллекций; поддержка хранения векторов на диске (on_disk=True), снижающая потребление оперативной памяти при росте корпуса; возможность как облачного, так и локального развёртывания через Docker-контейнер. При выборе Qdrant рассматривались альтернативы: Pinecone — исключён как проприетарный облачный сервис без локального развёртывания; Weaviate — отличается большей сложностью конфигурации при малом масштабе; Chroma — удобен для прототипирования, но уступает Qdrant по производительности при росте объёма коллекции. По совокупности критериев Qdrant признан оптимальным.

Учебные материалы Industrix представлены преимущественно в формате PDF. Для работы с ними реализована двухуровневая система обработки. Цифровые PDF обрабатываются библиотекой **pypdf** — лёгким Python-парсером, обеспечивающим постраничное извлечение текста с сохранением номеров страниц. Дополнительно используется **PyMuPDF** [19] — Python-обёртка над библиотекой MuPDF — для структурного анализа страниц: обнаружения табличных сеток по векторным примитивам, вычисления покрытия изображениями и рендеринга страниц в растровые изображения для последующего OCR.

Для отсканированных страниц реализована двухмаршрутная система OCR. Первый маршрут — **Tesseract OCR 5** [20] через обёртку pytesseract с языковым пакетом rus+eng — применяется для страниц с однородным текстовым содержанием. Tesseract реализует архитектуру LSTM и обеспечивает высокое качество распознавания при умеренных вычислительных требованиях. Второй маршрут — мультимодальная языковая модель (VLM) через локальный сервер **Ollama** [21] — применяется для страниц со сложной структурой: таблицами, многоколоночной вёрсткой, схемами. VLM значительно превосходит Tesseract по качеству восстановления структурированного содержания, поскольку понимает визуальный контекст изображения, а не только распознаёт отдельные символы. При недоступности Ollama реализован автоматический откат к Tesseract.

Пользовательский интерфейс реализован в виде **Telegram-бота** на библиотеке **python-telegram-bot 20.7**. Выбор Telegram обусловлен тем, что мессенджер уже установлен у подавляющего большинства целевой аудитории в России и СНГ, что устраняет барьер входа; Bot API предоставляет бесплатный и стабильный интерфейс; асинхронная архитектура библиотеки версии 20.x позволяет обрабатывать параллельные сессии

без значительных изменений в кодовой базе. Для векторных операций применяется **NumPy**, для асинхронных HTTP-запросов к Ollama — **httpx**, для структурированного описания состояний — **dataclasses** стандартной библиотеки. Основной язык разработки — Python 3.11.

При разработке системы в качестве вспомогательного инструмента использовался ИИ-ассистент Claude Sonnet (Anthropic). С его помощью генерировались шаблоны промптов для агентов (с последующей ручной доработкой под педагогические требования), вспомогательные функции (L2-нормализация, безопасный парсинг JSON, детектор незнания), а также прорабатывались варианты архитектурных решений, верифицировавшихся экспериментально. Все ключевые решения — архитектура агентов, пороговые значения фильтрации, структура промптов, педагогическая логика — разрабатывались и корректировались автором самостоятельно на основе результатов тестирования. Применение ИИ-ассистента не распространялось на этапы постановки исследовательских вопросов, интерпретации результатов экспериментов и формулировки выводов.

Сводная характеристика технологического стека представлена в таблице 2.3.

Таблица 2.3 — Технологический стек ИИ-тьютора

Уровень	Инструмент	Назначение
Языковая модель	GPT-4o-mini (OpenAI)	Генерация педагогических ответов, наводящих вопросов, оценок
Эмбединги	text-embedding-3-large	Векторизация чанков и запросов (3 072 измерения)
Векторная БД	Qdrant	Хранение и семантический поиск по корпусу
PDF-парсинг	pypdf, PyMuPDF	Извлечение текста и структурный анализ страниц
OCR (текст)	Tesseract 5 + pytesseract	Распознавание однородных отсканированных страниц
OCR (таблицы/схемы)	Ollama VLM	Распознавание страниц со сложной структурой
Интерфейс	python-telegram-bot 20.7	Telegram-бот для взаимодействия со студентами
Асинхронность	asyncio, httpx	Неблокирующая обработка запросов
Вычисления	NumPy	L2-нормализация эмбедингов
Язык разработки	Python 3.11	Основной язык реализации

2.3 Реализация MVP ИИ-тьютора

Система ИИ-тьютора реализована как многоагентный RAG-конвейер [22], в котором каждый входящий вопрос студента обрабатывается модулем семантического поиска и тремя специализированными агентами с чётко разделёнными педагогическими функциями. Взаимодействие со студентом опосредовано Telegram-ботом, выполняющим роль интерфейсного шлюза без какой-либо бизнес-логики.

Жизненный цикл одного обращения включает следующие шаги:

- 1) студент направляет вопрос в чат;
- 2) обработчик `handle_message` передаёт текст в оркестратор `TutorOrchestrator`;
- 3) оркестратор вызывает модуль семантического поиска `retrieve_context`, который векторизует вопрос через API эмбедингов и выполняет поиск по коллекции Qdrant с фильтром по пространству знаний;
- 4) найденные чанки передаются агенту-планировщику `PlannerAgent`, определяющему стратегию педагогического воздействия;
- 5) на основании стратегии агент-тьютор `TutorAgent` формирует педагогически обоснованный ответ, который отправляется студенту;
- 6) после получения реакции студента агент-оценщик `EvaluatorAgent` анализирует её и передаёт результат в следующую итерацию;
- 7) цикл повторяется до достижения критерия завершения сессии.

Всё взаимодействие с языковой моделью и векторной базой данных выполняется асинхронно через `asyncio run_in_executor`, что предотвращает блокировку event-loop при параллельных сессиях.

Ключевым архитектурным решением является принцип разделения ответственности между агентами: планировщик отвечает за *что делать*, тьютор — за *как сформулировать*, оценщик — за *как интерпретировать ответ студента*. Такое разделение упрощает отладку: при нарушении педагогического поведения системы

значительно проще локализовать, какой агент функционирует некорректно. Кроме того, каждый агент может независимо настраиваться и заменяться без влияния на остальные компоненты. Альтернативный подход — единый монолитный промпт, совмещающий все функции — был опробован на ранних этапах разработки и отклонён: изменение одной части промпта непредсказуемо влияло на поведение других функций, а локализация ошибок была крайне затруднена.

Прежде чем тьютор сможет отвечать на вопросы, необходимо однократно проиндексировать учебный корпус. Конвейер индексации реализован в двух версиях: прототипная создана в Jupyter Notebook в ходе экспериментов, производственная оформлена в виде модулей `rag_core.py` и `ocr.py` с полноценной обработкой ошибок, кэшированием и поддержкой идиоматичного обновления корпуса.

Конвейер индексации начинается с анализа каждого PDF-файла. Функция `analyze_pdf` открывает документ через PyMuPDF и для каждой страницы применяет два классификатора. Первый — `classify_scan_type` — определяет, является ли страница цифровой или отсканированной, на основе соотношения числа реально извлечённых символов к ожидаемому для страницы данной площади: если отношение ниже порога `SCAN_TEXT_THRESHOLD`, страница считается отсканированной. Второй — `classify_complexity` — оценивает структурную сложность содержимого. Таблица обнаруживается функцией `_detect_grid` по наличию не менее трёх горизонтальных и трёх вертикальных векторных линий в примитивах страницы. Страница с покрытием изображениями более 80% и менее 50 символами классифицируется как `image_only`; с покрытием более 30% при наличии текста — как `mixed`; с более чем двумя шрифтами — как `formatted_text`; остальные — как `simple_text`. По результатам анализа страницам назначается OCR-маршрут: сложные страницы (`table`, `mixed`, `formatted_text`) направляются к VLM, простые отсканированные — к Tesseract. Реализация классификации приведена в листинге 2.1.

```
1 def classify_scan_type(page: fitz.Page) -> str:
2     text = page.get_text("text").strip()
3     char_count = len(text)
4     page_area = page.rect.width * page.rect.height
5     expected = _EXPECTED_CHARS_PER_A4 * (page_area / _A4_AREA)
6     ratio = char_count / expected if expected > 0 else 1.0
7     return "digital" if ratio >= config.SCAN_TEXT_THRESHOLD else "scanned"
8
9
10 def classify_complexity(page: fitz.Page) -> str:
11     if _detect_grid(page):
12         return "table"
13     image_coverage = _compute_image_coverage(page)
14     char_count = len(page.get_text("text").strip())
15     if image_coverage > 0.8 and char_count < 50:
16         return "image_only"
17     if image_coverage > 0.3 and char_count > 50:
18         return "mixed"
19     if _count_fonts(page) > 2:
20         return "formatted_text"
21     return "simple_text"
22
23
24 def _detect_grid(page: fitz.Page) -> bool:
25     drawings = page.get_drawings()
26     horiz = vert = 0
27     for d in drawings:
28         for item in d.get("items", []):
29             if item[0] == "l":
30                 p1, p2 = item[1], item[2]
```

```

31         dx, dy = abs(p2.x - p1.x), abs(p2.y - p1.y)
32         if dx > 20 and dy < 3:
33             horiz += 1
34         elif dy > 20 and dx < 3:
35             vert += 1
36     return horiz >= 3 and vert >= 3

```

Listing 2.1: Классификация типа и сложности страниц PDF

После анализа извлечённый текст разбивается на чанки методом скользящего окна с размером 400 слов и перекрытием 50 слов. Эти параметры подобраны экспериментально: при 200 словах чанки теряют смысловой контекст на границах, при 600 — становятся семантически неоднородными, что снижает точность поиска. Перекрытие в 50 слов гарантирует, что ключевые фразы на границах двух чанков присутствуют в обоих и не выпадают из поиска. Каждому чанку присваивается порядковый идентификатор `chunk_id` и метаданные: номер страницы, имя файла-источника и идентификатор пространства знаний.

Чанки векторизуются функцией `embed_batch` батчами по 64 элемента через API OpenAI. К каждому вектору применяется L2-нормализация. Нормализованные векторы вместе с полезной нагрузкой загружаются в Qdrant методом `upsert`. При повторной индексации предварительно вызывается `delete_chunks_by_source`, удаляющая устаревшие чанки — это обеспечивает идемпотентность конвейера и исключает дублирование при обновлении корпуса. Производственная версия включает также механизм миграции данных на основе `sentinel`-файлов: при каждом обновлении конвейера (например, при изменении параметров нормализации или чанкинга) миграция выполняется ровно один раз и не повторяется благодаря проверке файла-маркера.

Для снижения числа повторных обращений к API реализован двухуровневый кэш. Первый уровень — кэш эмбедингов: функция `embed_batch` перед запросом к API проверяет наличие результата через `cache.get_embeddings_batch` и формирует запрос только для отсутствующих текстов. Второй уровень — кэш завершений: функция `ask_openai` использует пару (системный промпт, вопрос) как ключ и при совпадении возвращает сохранённый ответ без обращения к языковой модели. Это существенно снижает задержку и стоимость при повторных идентичных вопросах — типичная ситуация для образовательного сервиса, где студенты независимо задают одни и те же вопросы по общим темам.

Центральным компонентом производственной реализации является функция `retrieve_context` в модуле `rag_core.py`, принимающая текст вопроса, максимальное число чанков `top_k` (по умолчанию 10) и идентификатор пространства знаний `space` (по умолчанию "default"). Функция векторизует вопрос с кэшированием, выполняет поиск в Qdrant с фильтром по пространству знаний и применяет двухуровневую фильтрацию по косинусному сходству, реализованную следующим образом.

Алгоритм работы функции состоит из следующих шагов:

- 1) вопрос векторизуется через `embed_text` (с кэшированием);
- 2) выполняется поиск в Qdrant с фильтром по полю `space` и ограничением `top_k` по числу результатов;
- 3) из результатов отфильтровываются пустые чанки;
- 4) применяется двухуровневая фильтрация по порогу косинусного сходства.

```

1 #
2 filtered = [c for c in contexts
3             if c["score"] >= config.SCORE_THRESHOLD_PRIMARY]
4
5 if not filtered:
6     #
7     filtered = [c for c in contexts
8                 if c["score"] >= config.SCORE_THRESHOLD_FALLBACK]
9
10 #             , filtered
11 # ->         [NO_ANSWER]

```

Listing 2.2: Двухуровневая фильтрация результатов поиска

Двухуровневая схема фильтрации решает следующую задачу. Высокий основной порог отсекает нерелевантные чанки и обеспечивает высокую точность при стандартных запросах. Однако при перефразированных вопросах — где студент выражает ту же мысль иначе — косинусное сходство может оказаться умеренным: ниже основного, но всё же достаточным для корректного ответа. Именно для таких случаев введён резервный порог: он позволяет системе найти релевантный контекст и ответить, не демонстрируя студенту отказ от темы, которая в материалах реально присутствует. Если же и резервный порог не достигнут — вопрос действительно выходит за пределы корпуса, и система возвращает маркер [NO_ANSWER], явно информируя студента об отсутствии соответствующих материалов.

Функция `build_prompt` формирует системный промпт для языковой модели. Каждый чанк снабжается атрибуцией в формате [. N | : < >], позволяющей агенту-тьютору ссылаться на конкретный раздел материалов при объяснении. Промпт содержит явную инструкцию: при недостаточности информации в контексте начать ответ с фиксированного маркера [NO_ANSWER]. Использование машиночитаемого маркера — вместо свободного текстового ответа — обеспечивает надёжное программное обнаружение ситуации «ответа нет» без дополнительного NLP-анализа текста ответа.

Состояние учебной сессии инкапсулировано в датаклассе `StudentState` со следующими полями:

- `topic` — тема текущего вопроса;
- `attempts` — счётчик попыток студента;
- `mastery_score` — накопленный балл освоения (0–1);
- `history` — список реплик диалога в хронологическом порядке;
- `misconceptions` — список выявленных заблуждений;
- `last_strategy` — стратегия, применённая на предыдущем шаге;
- `explanation_used` — флаг применения объяснения.

Оркестратор `TutorOrchestrator` управляет жизненным циклом сессии: инициализирует состояние при создании, запускает основной цикл взаимодействия и завершает сессию при достижении критерия окончания. Критерии завершения: активация стратегии `explain` (объяснение выдано — дальнейший опрос нецелесообразен) или достижение лимита попыток (по умолчанию 5 итераций). Многоагентная педагогическая логика реализована в виде трёх специализированных классов, управляемых оркестратором `TutorOrchestrator`. Состояние учебной сессии инкапсулировано в датаклассе `StudentState` со следующими полями:

- `topic` — тема текущего вопроса;
- `attempts` — счётчик попыток студента;
- `mastery_score` — накопленный балл освоения (0–1);
- `history` — список реплик диалога в хронологическом порядке;
- `misconceptions` — список выявленных заблуждений;
- `last_strategy` — стратегия, применённая на предыдущем шаге;
- `explanation_used` — флаг применения объяснения.

Полный вид датакласса приведён в листинге 2.3.

```
1 @dataclass
2 class StudentState:
3     topic: str
4     attempts: int = 0
5     mastery_score: float = 0.0
6     history: List[str] = field(default_factory=list)
7     misconceptions: List[str] = field(default_factory=list)
8     last_strategy: Optional[str] = None
9     explanation_used: bool = False
```

Listing 2.3: Датакласс StudentState — состояние учебной сессии

Агент-планировщик `PlannerAgent` принимает решение о стратегии на каждой итерации. Его промпт включает: текущий вопрос, форматированный контекст из материалов, все поля объекта `StudentState`, последний ответ студента и результат его оценки. Агент запрашивает у GPT-4o-mini стратегию в строгом формате JSON: `{"strategy": "socratic|hint|explain"}`. Использование JSON-вывода — вместо свободного текста — обеспечивает надёжное ветвление управляющей логики. Для надёжного парсинга применяется функция `parse_json`, удаляющая возможные Markdown-обёртки перед десериализацией. Правила выбора стратегии, закреплённые в промпте, таковы: при первом вопросе без истории — `socratic`; при частично верном ответе на первой попытке — `hint`; при двух и более неудачных попытках — `explain`; при оценке выше 0,8 — снова `socratic` с усложняющим вопросом.

Правила выбора стратегии закреплёны в промпте:

- если студент ответил правильно (балл освоения более 0,8) — перейти к усложняющему вопросу в режиме `socratic`;
- если студент явно не понял тему или ответил частично верно при первой попытке — применить `hint`;
- если число попыток достигло двух и более, либо уже применялись подсказки без результата — перейти к `explain`;
- не повторять одну стратегию бесконечно, избегать «зависания» в сократическом режиме.

Использование JSON-вывода планировщика обеспечивает строгое ветвление логики оркестратора и устраняет неопределённость, характерную для свободного текстового вывода. Для надёжного парсинга применяется функция `parse_json`, удаляющая возможные Markdown-обёртки (````json...````) перед десериализацией.

Дополнительно реализована функция `is_no_knowledge_answer` — детектор явного незнания, анализирующий ответ студента на наличие триггерных фраз: «не знаю», «не читал», «без понятия», «затрудняюсь», «не уверен», «не помню». При обнаружении любой из них оркестратор немедленно переключается на стратегию `explain`, минуя сократический этап. Это предотвращает педагогически нецелесообразную ситуацию, при которой студент без базовых знаний вынужден отвечать на наводящие вопросы.

Агент-тьютор `TutorAgent` генерирует непосредственный педагогический ответ. Его промпт параметризован стратегией. В режиме `socratic` агент задаёт единственный наводящий вопрос, не раскрывающий ответа; в режиме `hint` — краткую подсказку объёмом не более двух предложений; в режиме `explain` — полное объяснение с обязательным указанием источника в формате [: < >, . <N>]. Во всех режимах агент ограничен предоставленным контекстом: промпт содержит явный запрет на использование знаний вне контекста и изобретение источников. Фрагмент реализации параметризации промпта приведён в листинге 2.4.

```
1 # Each strategy gets its own instruction block
2 mode_instruction = {
3     TutorMode.SOCRATIC: (
4         "Ask ONE leading question. Do NOT reveal the answer.\n"
5         "Use ONLY the context provided below.\n"
6         "Forbidden: outside knowledge, invented sources."
7     ),
8     TutorMode.HINT: (
9         "Give a short hint (1-2 sentences). "
10        "Do NOT reveal the full answer. Context only."
11    ),
12    TutorMode.EXPLAIN: (
13        "Explain the topic briefly (4-6 sentences) from context.\n"
14        "Cite source: [Source: SOURCE, p. PAGE]"
15    ),
16 }[mode]
```

```

17
18 prompt = f"""
19 You are an AI tutor for the Industrix course.
20 {mode_instruction}
21
22 CONTEXT:
23 {context_text}
24
25 DIALOGUE HISTORY:
26 {history_text}
27
28 STUDENT QUESTION: {question}
29
30 ANSWER:
31 """

```

Listing 2.4: Tutor agent: prompt parametrized by strategy

Агент-оценщик анализирует ответ студента и возвращает структурированную оценку в формате JSON со следующими полями:

- `correct` — булево значение, отражающее абсолютную правильность;
- `partial` — признак частично верного ответа;
- `misconception` — текстовое описание обнаруженного заблуждения (пустая строка при отсутствии);
- `score` — числовая оценка от 0 до 1, передаваемая планировщику для принятия решения о стратегии.

Диалоговый цикл завершается при применении стратегии `explain` или при достижении лимита попыток (по умолчанию 5 итераций). Оба условия предотвращают бесконечный цикл.

Результат оценки включается в контекст следующего обращения к планировщику, формируя замкнутый цикл педагогической обратной связи. Выявленные заблуждения сохраняются в поле `misconceptions` состояния студента, что потенциально позволяет в будущих версиях адаптировать объяснения с учётом конкретных ошибок студента.

Обработка OCR в модуле `ocr.py` реализована как асинхронная функция `process_pdf`, итерирующая по страницам документа, пропуская цифровые и направляя отсканированные к соответствующему маршруту. Для предотвращения ошибок выделения памяти функция `_safe_zoom` динамически снижает DPI при превышении лимита в 50 миллионов пикселей. При ошибках на стороне Ollama — `TimeoutException`, `ConnectError`, `HTTPStatusError` — функция `vlm_ocr` автоматически откатывается к Tesseract с соответствующей пометкой в результате. Статистика OCR аккумулируется в объекте `IngestResult`: поля `ocr_pages` и `vlm_pages` позволяют мониторить распределение нагрузки, `warnings` — отслеживать страницы с ошибками распознавания.

Итоговый промпт-инжиниринг системы строится на нескольких принципах, последовательно применённых ко всем агентам. Первый — ограничение контекстом: каждый промпт содержит явный запрет на использование знаний вне предоставленного контекста и прямое указание сообщать об отсутствии информации. Второй — структурированный вывод: там, где требуется машиночитаемый результат (планировщик, оценщик), языковая модель получает инструкцию вернуть строго JSON без комментариев и обёрток. Третий — минимизация неопределённости: каждая стратегия описана через чёткие запреты и обязательства («задай ОДИН вопрос», «не более двух предложений», «укажи источник»), а не через размытые рекомендации. Четвёртый — включение состояния: промпты планировщика и тьютора включают историю диалога, что позволяет языковой модели учитывать динамику взаимодействия. Реализация этих принципов потребовала многократного итеративного тестирования: на ранних версиях планировщик нередко возвращал свободный текст вместо JSON, тьютор раскрывал ответ даже в режиме `socratic`, а оценщик классифицировал все ответы как верные. Каждая из этих проблем устранялась корректировкой соответствующего фрагмента промпта по результатам тестирования.

Производственная версия системы включает механизм миграции данных, реализованный через функции `migrate_collection` и `migrate_spaces`. Идемпотентность обеспечивается через `sentinel`-файлы: при первом запуске функция выполняет миграцию и записывает файл-маркер; при последующих запусках наличие маркера сигнализирует о завершённости миграции. Это позволяет безопасно обновлять конвейер индексации (например, при изменении функции нормализации или параметров чанкинга) без риска повторного применения миграции.

2.4 Анализ результатов

Оценка разработанного ИИ-тьютора проводилась в два взаимодополняющих этапа: техническое тестирование RAG-конвейера и педагогическое тестирование многоагентной логики. Техническое тестирование было направлено на измерение точности семантического поиска, корректности двухуровневой фильтрации, поведения системы при выходе запроса за пределы корпуса и производительностных характеристик. Педагогическое тестирование оценивало соответствие действий агентов сценариям обучения: выбор стратегий планировщиком, точность оценки ответов студента и надёжность детектора незнания. Методология оценки опиралась на ручной анализ результатов автором работы; внедрение автоматических метрик (RAGAS [es2023ragas], MRR, NDCG) определено как задача следующего этапа разработки. Это ограничение необходимо учитывать при интерпретации результатов: ручная оценка субъективна и чувствительна к эффекту подтверждения ожиданий.

Тестирование проводилось на корпусе учебных материалов компании Industrix — PDF-презентациях по предпринимательству, запуску стартапов и бизнес-моделированию. Корпус содержал как цифровые документы, так и отсканированные материалы с таблицами и схемами. Коллекция Qdrant создавалась с нуля в ходе работы над проектом; суммарный объём проиндексированного корпуса составил несколько тысяч текстовых чанков, извлечённых из документов общим числом страниц свыше ста.

Для оценки качества семантического поиска сформирована тестовая выборка из 30 вопросов, разделённых на три группы по 10 вопросов. Первая группа — прямые вопросы в терминологии учебных материалов (например: «Какие причины провала стартапа?», «Из чего состоит бизнес-модель?», «Что такое MVP в контексте запуска продукта?»). Вторая группа — перефразированные вопросы с иной лексикой: те же темы, что и в первой группе, но выраженные синонимами или описательными конструкциями. Третья группа — вопросы полностью вне учебного корпуса (медицина, законодательство, технические области, не представленные в материалах): ожидаемый результат — маркер `[NO_ANSWER]`, а не ответ из общих знаний модели. Результаты тестирования по группам представлены в таблице 2.4.

Таблица 2.4 — Результаты тестирования семантического поиска

Группа вопросов	Вопросов	Корректный результат	Точность
Прямые вопросы (терминология корпуса)	10	9	90%
Перефразированные вопросы	10	8	80%
Вопросы вне корпуса	10	5	50%
Итого	30	22	73%

Система корректно обработала практически большинство прямых вопросов: по каждому из них в топ-10 результатов попадали чанки с оценкой выше основного порога. Единственная неудача в группе перефразированных вопросов связана с запросом об «отрицательном денежном потоке», тогда как в корпусе эта тема освещалась через термин «кассовый разрыв». Семантическое расстояние между этими понятиями в пространстве `text-embedding-3-large` оказалось выше обоих порогов фильтрации, и система корректно вернула маркер `[NO_ANSWER]`: с формальной точки зрения поведение правильное, однако педагогически нежелательное — тема в материалах присутствует, просто под другим именем. Данный случай указывает на необходимость расширения корпуса синонимами или применения методов расширения запроса. Для третьей группы система во всех 10 случаях корректно отказала в ответе, не обратившись к общим знаниям модели — критически важный результат с точки зрения педагогической достоверности системы.

Для оценки вклада двухуровневой фильтрации проведён контрольный эксперимент: та же выборка из 30 вопросов обрабатывалась без резервного порога — только с основным. В этом случае дополнительно 3 перефразированных вопроса давали пустой результат при реально имеющейся в корпусе информации. Точность по второй группе снизилась с 90 до 60%. Введение резервного порога увеличило полноту без снижения точности: все дополнительно найденные чанки были действительно релевантны.

В части производительности измерялось время отклика системы — от получения сообщения Telegram до отправки ответа — при однопользовательской нагрузке. Среднее значение составило 3–5 секунд. Доминирующий вклад в задержку вносит обращение к OpenAI API: векторизация запроса занимает 300–500 мс при промахе кэша и менее 5 мс при попадании, генерация ответа — 2–4 секунды. Поиск в Qdrant при объёме коллекции в несколько тысяч чанков занимал 50–150 мс — незначимый вклад. Кэширование эмбедингов и завершений ускоряло повторные идентичные запросы практически до мгновенного ответа. OCR Tesseract обрабатывал страницу А4 за 2–4 секунды при 300 DPI; VLM через Ollama — 10–20 секунд в зависимости от аппаратного обеспечения. Характеристики производительности по компонентам приведены в таблице 2.5.

Таблица 2.5 — Характеристики задержки по компонентам системы

Компонент	Задержка, мс	Условие
Векторизация запроса (API)	300–500	Промах кэша
Векторизация запроса (кэш)	< 5	Попадание в кэш
Поиск в Qdrant	50–150	~несколько тыс. чанков
Генерация ответа GPT-4o-mini	2000–4000	200–400 токенов
Генерация ответа (кэш)	< 5	Попадание в кэш
Отправка через Telegram API	100–300	Зависит от сети
OCR Tesseract (страница А4)	2000–4000	300 DPI
OCR VLM Ollama (страница А4)	10000–20000	Зависит от GPU

Педагогическое тестирование проводилось посредством имитации диалогов по шести заранее подготовленным сценариям, охватывающим основные ветки управляющей логики оркестратора. Результаты приведены в таблице 2.6.

Таблица 2.6 — Результаты тестирования педагогических стратегий агента-планировщика

Сценарий	Ожид. стратегия	Результат	Примечание
Первый вопрос без истории	socratic	Верно	Базовый сценарий
Частично верный ответ, 1-я попытка	hint	Верно	Типичный сценарий
Неверный ответ, 2-я попытка	explain	Верно	Порог попыток
Ответ содержит «не знаю»	explain (детектор)	Верно	Быстрый путь
Правильный ответ (score > 0,8)	socratic (усложн.)	Верно	Прогрессия
Вопрос вне корпуса	Маркер [NO_ANSWER]	Верно	Отказ от ответа

Агент-планировщик корректно выбирал стратегию во всех шести сценариях. Принципиально важным результатом является отсутствие «зависания» в бесконечном сократическом цикле: ни в одном из тестов система не задала более двух последовательных наводящих вопросов без перехода к подсказке или объяснению. Детектор незнания срабатывал корректно для всех протестированных формулировок из зафиксированного набора триггеров, однако не тестировался на нестандартных вариантах перефразирования — это известное ограничение текущей реализации.

Агент-оценщик тестировался на 20 парах «вопрос — ответ студента»: 10 пар с верными ответами, 5 с частично верными и 5 с неверными. Точность классификации по полю correct составила 18 из 20 (90%). В двух граничных случаях агент посчитал частично верный ответ полностью верным: ответ содержал правильную суть, но упускал существенную деталь, которую агент расценил как несущественную. Поле score демонстрировало разумное ранжирование: верные ответы получали 0,7–1,0; частично верные — 0,3–0,7; неверные — 0,0–0,3. JSON-вывод парсировался корректно во всех 20 случаях.

В ходе разработки была проведена серия экспериментов с параметрами разбиения на чанки, результаты которых представлены в таблице 2.7.

Таблица 2.7 — Влияние параметров чанкинга на точность семантического поиска

Размер окна, слов	Перекрытие, слов	Точность	Наблюдение
200	50	73%	Потеря контекста на границах; ответы неполны
400	50	90%	Оптимальный баланс контекста и однородности
600	50	82%	Семантическая неоднородность чанков; поиск менее точен

При $\text{top}_k=5$ ответы были более сфокусированы, однако для многоаспектных вопросов, ответы на которые рассредоточены по нескольким документам, в контекст не попадали важные дополнительные сведения. При $\text{top}_k=10$ промпт увеличивался примерно на 2000 токенов, повышая стоимость запроса, однако полнота ответов для сложных вопросов возрастала. Значение $\text{top}_k=10$ принято как рабочее для данного корпуса.

Параллельно в ходе разработки последовательно реализованы два режима — базовый RAG-ответчик и многоагентный тьютор, — позволившие провести качественное сравнение подходов, результаты которого представлены в таблице 2.8.

Таблица 2.8 — Сравнение базового RAG-ответчика и многоагентного тьютора

Характеристика	Базовый RAG	Многоагентный тьютор
Педагогическая ценность	Низкая	Высокая
Число обращений к LLM на вопрос	2	5–8
Средняя задержка ответа, с	3–4	4–8
Соблюдение границ корпуса	Да	Да
Адаптация к ответам студента	Нет	Да
Выявление заблуждений	Нет	Да
Стоимость инференса (относит.)	1,0	3,5–4,0
Сложность реализации и отладки	Низкая	Средняя

Базовый RAG-ответчик целесообразен как режим быстрого поиска по материалам; многоагентный тьютор обеспечивает принципиально иное педагогическое качество за счёт диалоговой природы взаимодействия и адаптации к ответам студента. В рамках MVP многоагентный режим является основным; базовый режим сохранён как запасной на случай технических неполадок с цепочкой агентов.

Анализ реализации выявил четыре существенных ограничения текущего MVP. Первое — отсутствие персистентного хранения истории: каждое сообщение в Telegram инициирует новую независимую сессию без памяти о предыдущих взаимодействиях. Студент, возвращающийся к теме спустя время, не получает непрерывного обучения. Устранение требует интеграции хранилища сессий (Redis или PostgreSQL) с привязкой к идентификатору пользователя Telegram. Второе — ручная методология оценки без привлечения реальных студентов и автоматических метрик. Инструментальная оценка через RAGAS [23] — набор метрик для RAG-систем, включающий точность ответов, полноту извлечения и релевантность контекста — позволила бы получить объективные воспроизводимые результаты. Третье — простота детектора незнания: текущая реализация через точное вхождение фраз чувствительна к орфографии и не покрывает нестандартных формулировок («я не в курсе», «понятия не имею», «сложно сказать»). Более надёжным решением была бы zero-shot классификация намерений через языковую модель или специализированный классификатор. Четвёртое — отсутствие мониторинга: система не собирает операционных метрик — распределение выбранных стратегий, доля ответов с маркером [NO_ANSWER], средний балл освоения по темам. Без мониторинга невозможно принимать обоснованные решения о доработке системы.

По результатам реализации и тестирования MVP можно констатировать следующее. Разработанная система демонстрирует работоспособность всех заявленных компонентов: конвейер индексации успешно обработал смешанный корпус с применением гибридного OCR, семантический поиск обеспечил точность 97% на тестовой выборке, многоагентная логика корректно реализует сократический диалог с адаптацией к ответам студента. Вместе с тем достигнутые результаты носят преимущественно технический характер. Педагогическая эффективность — в какой мере сократический диалог реально улучшает усвоение материала — может быть оценена только в ходе контролируемого пользовательского исследования с реальными

студентами и измерением образовательных результатов. Именно это исследование определяет основное направление следующего этапа работы.

Выявленные ограничения — отсутствие персистентной истории, ручная оценка качества, примитивный детектор намерений, отсутствие мониторинга — формируют конкретный план следующего этапа: интеграция хранилища сессий, автоматические метрики RAGAS, нейронная классификация педагогических состояний и система операционного мониторинга. Реализация этих направлений создаст основу для полноценного продуктового тестирования системы в образовательной среде Industrix.

3 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ

В настоящей главе описаны результаты инженерной части работы по созданию интеллектуального тьютора Smart Line — сквозного AI-помощника студента в образовательной платформе Adapstory. Раздел 3.1 посвящён целевой (продакшн) архитектуре тьютора: его месту в платформе, пяти ролевым зонам (Student Lifecycle Spaces), декомпозиции на микросервисы, модели сессии, жизненному циклу виджета, пяти уровням взаимодействия, оркестрации семи агентов, асинхронным интеграциям, инфраструктурным требованиям и подходу к наблюдаемости. Раздел 3.2 описывает реализованный MVP-прототип: выбранный стек, перенос семи агентов из исследовательского Jupyter-ноутбука в FastAPI-сервис, контракт HTTP-API, механизм проекции виджетов, фронтенд с автоматическим переключением между реальным и мок-бэкендом, а также перечень осознанных ограничений MVP и явное сопоставление каждого из них с путём переноса в промышленную эксплуатацию.

3.1 Архитектура продакшн решения

Smart Line спроектирован как универсальная точка входа студента во все AI-возможности платформы. Это не только тьютор учебного содержания, но и помощник по организационным вопросам курса, навигации по каталогу, рефлексии прогресса и настройкам профиля. В основе архитектуры лежат три инженерных принципа:

- 1) **GenAI-native** — большая языковая модель (LLM) рассматривается как основной исполнительный компонент, а не вспомогательный API. Любая сквозная функция тьютора описывается в терминах четырёх измерений: роль LLM в сценарии, механизмы управляемости вывода, контроль знаний (RAG и граф компетенций), измеримый образовательный эффект.
- 2) **Plugin-first** — ядро платформы минимально и состоит из нескольких базовых ограниченных контекстов; конкретные тьюторские возможности поставляются плагинами, декларирующими набор шаблонов виджетов, допустимых действий студента и версионированных промптов.
- 3) **Гексагональная декомпозиция** — внутри каждого ограниченного контекста поддерживается строгое разделение доменного слоя, прикладных сценариев и адаптеров. Доменный слой не зависит ни от Spring, ни от jOOQ, ни от Kafka SDK; все внешние обращения выполняются через порты-интерфейсы.

Smart Line представлен на фронтенде как плавающая нижняя панель, доступная в любой точке интерфейса студента. Панель принимает текстовый или голосовой запрос, показывает чипы пяти зон (Spaces), а по итогам обращения регенерирует на экране Widget Lake— адаптивную сетку виджетов в стиле Bento, отражающую актуальное состояние обучения. На контекстном уровне (C4 Level 1) Smart Line связывает студента с ядром платформы через BFF, опирается на шесть внешних сервисов (LLM-провайдеры через единый шлюз, векторное хранилище RAG, граф компетенций, реляционное хранилище транзакционного состояния, распределённый кэш, брокер событий). Схема представлена на рисунке 3.1.

Проектирование подчинено осознанному разделению ответственностей: маршрутизация намерений, состояние сессии, возможности плагинов, черновая генерация LLM, валидация политик и финальная проекция DivKit— различные ответственности, каждая реализована отдельным компонентом. Именно такая декомпозиция делает Smart Line безопасным, многоарендным, аудируемым и совместимым с уже существующим контуром «BFF → BC-16 → DivKit-рендерер».

Продуктовые цели Smart Line выходят за рамки одного тьютора:

- сопровождение обучения (объяснения, проверка понимания, адаптивные вопросы);
- ответы на организационные вопросы курса (дедлайны, расписание, форматы сдачи);
- навигация по каталогу и обнаружение релевантных продуктов;
- рефлексия прогресса (сильные и слабые темы, рекомендуемые действия);
- помощь с профилем, настройками и онбордингом.



c4-l1-smart-line.png

Рисунок 3.1 — Контекстная диаграмма Smart Line

Технические ограничения зафиксированы заранее:

- весь клиентский трафик идёт только через BFF, прямые вызовы из браузера в бэкэнд-контексты запрещены;
- первичная JS-полезная нагрузка не превышает действующий фронтенд-бюджет (sub-250 КБ);
- виджеты рендерятся только в формате DivKit JSON через существующий рендерер;
- действия над виджетами выполняются через контролируемый контракт `custom_action`;
- изоляция арендатора обязательна по HTTP, Redis, Kafka и всем слоям персистентности;
- базовый уровень доступности интерфейса — WCAG 2.1 AA.

Осознанные не-цели первой версии: свободная генерация произвольного UI языковой моделью, полноценный L5-чат в каждом запросе, общая Smart Line-память между арендаторами, ранжирование на основе ML-моделей за пределами детерминированных правил.

При проектировании приняты следующие фундаментальные решения:

- 1) **Единая точка входа.** Smart Line является единственным входом студента ко всем AI-возможностям, как внутри активного урока, так и вне его. Это исключает дублирование слоя LLM-оркестрации по разным экранам.
- 2) **Пять зон с двойной ролью.** Набор LEARNING, ANALYTICS, MARKETPLACE, PROFILE, ONBOARDING фиксирован как продуктовое понятие и одновременно используется сервером как контекст маршрутизации намерений и контекст политик плагинов. Арендаторы могут настраивать видимость и названия зон, но не их семантическое значение.
- 3) **Маршрутизация намерений выполняется до тьюторского конвейера.** Каждый запрос Smart Line сначала проходит через Intent Router и только затем, в случае LEARNING_UNDERSTANDING, попадает в тьюторский оркестратор. Это исключает применение сократической стратегии к организационным вопросам и освобождает LLM-бюджет.
- 4) **LLM порождает черновики, а не исполняемый UI.** Языковая модель возвращает типизированный WidgetDraft с валидируемым контентом; финальный DivKit JSON собирает серверный Projection Builder после прохождения Policy Gate. Это устраняет класс атак prompt injection на уровне UI.
- 5) **Источник истины.** Владелец жизненного цикла Widget Lake является BC-16 Personalization Runtime: плагины декларируют возможности и шаблоны, LLM предлагает содержимое, а BC-16 решает, что действительно публикуется студенту.

На уровне контейнеров Smart Line состоит из компонентов, сведённых в таблицу 3.1.

Таблица 3.1 — Компоненты Smart Line и их ответственности

Компонент	Ответственность	Технология
Smart Line UI	сбор текстового и голосового ввода, переключение зон, рендер Widget Lake, отправка действий	Next.js 16 + DivKit + Web Speech
BFF Student	аутентификация, валидация запросов, проброс контекста арендатора и пользователя, формирование API	Spring WebFlux
BC-16 Personalization Runtime	жизненный цикл сессии, маршрутизация, оркестрация, публикация проекций	Spring Boot 4
Intent Router	rule-first маршрутизация, LLM-fallback классификация, владение QuestionType-Classifier	компонент BC-16
TutorOrchestratorAdapter	только learning: стратегии socratic / hint / explain / verify, делегирует Python-оркестратору	компонент BC-16
Plugin Gateway	поиск возможностей плагинов, здоровье провайдеров, тайм-аут и circuit-breaker	Spring Cloud Circuit-Breaker
LLM Gateway	прокси моделей, бюджет токенов, шаблоны промптов, валидация схем выхода	FastAPI + Pydantic
Policy Gate	валидация черновиков виджетов и allowlist действий перед проекцией	компонент BC-16
Projection Builder	преобразование типизированного черновика в итоговый DivKit JSON с Bento-подсказками	компонент BC-16
Smart Line AI Orchestrator (Python)	RAG-поиск + мультиагентный конвейер тьюторинга	FastAPI, Python 3.12
Neo4j (vector index)	векторный поиск по контенту курса и транскриптам уроков	Neo4j 5.x
Redis	кэш сессии арендатора и горячие проекции виджетов	Redis
PostgreSQL	долгосрочный аудит, метаданные шаблонов, настройки зон	существующие схемы
Kafka	CloudEvents о рендере, интеракциях, освоении темы, лайфцикле сессии	Apache Kafka

Компоненты Smart Line сознательно разделены между Java-стеком (BC-16, Policy Gate, Projection Builder, BFF) и Python-стеком (Smart Line AI Orchestrator, LLM Gateway). Java-слой отвечает за многоарендность, аудит, транзакционную целостность и интеграцию с существующими доменными сервисами Adapstory; Python-слой — за ML-операции, где нужна асинхронная работа с LLM, эмбедингами и графом агентов. Взаимодействие между стеками выполняется строго по REST и Kafka: прямой доступ Python к БД Java-контекста запрещён — это принцип «1 микросервис = 1 БД = 1 схема».

Каждая из пяти зон имеет собственный базовый набор виджетов и собственный набор приоритетных намерений; BC-16 учитывает текущую зону при маршрутизации, ретривале, ранжировании и выборе доступных возможностей. Распределение приведено в таблице 3.2.

Таблица 3.2 — Зоны жизненного цикла и их виджеты

Зона	Назначение	Приоритетные намерения	Типовые виджеты
LEARNING	объяснения, оценка, рекомендации	LEARNING_UNDERSTANDING	AskUserQuestion, ExplanationWidget, SkillProgress
ANALYTICS	прогресс, тренды, достижения	PROGRESS_REFLECTION	ProgressSummary, SkillRadar, AchievementBadge
MARKETPLACE	обнаружение офферов, подписки	NAVIGATION_DISCOVERY	ProductCard, CartSummary, RecommendedProducts
PROFILE	настройки, сертификаты	ACCOUNT_PROFILE, ORGANIZATIONAL	ProfileCard, Preferences-Widget
ONBOARDING	первые шаги, пререквизиты	NAVIGATION_DISCOVERY, ORGANIZATIONAL	OnboardingStep, QuickTour

Переключение зон по инициативе пользователя разрешено, но автоматическое «тихое» переключение по инициативе сервера в первой версии запрещено: система может только предложить студенту перейти в другую зону карточкой подсказки.

Каждый виджет объявляет один первичный уровень взаимодействия:

- **L1 Display** — только отображение; пример: SkillProgress показывает шкалу усвоения без возможности действия;
- **L2 Navigate** — переход к внешнему ресурсу или в другую зону; пример: RecommendedContent со ссылкой на статью;
- **L3 In-Widget Action** — действие внутри виджета без регенерации Widget Lake; пример: AskUserQuestion с открытым полем ответа;
- **L4 Regeneration** — действие вызывает полную регенерацию Widget Lake; пример: продолжение ответа студента после ExplanationWidget приводит к порождению следующего AskUserQuestion;
- **L5 Skill Launch** — запуск продолжительной сессии практики с ментором или углублённого сценария; в MVP отсутствует, предусмотрен для пост-MVP.

Объявление L4 возможно только если манифест плагина содержит в списке supportedInteractionLevels— это исключает неконтролируемое каскадное обновление проекций.

Сессия студента хранится в Redis с составным ключом `widget-lake:tenant:{tenant_id}:learner:{learner_id}:session:{session_id}` и TTL 1800 сек с автопродлением при активности (*sliding TTL*). Полезная нагрузка — JSON с фиксированной схемой. Ключевые поля:

- `session_id`, `tenant_id`, `learner_id` — идентификаторы сессии, арендатора и студента;
- `active_space` — текущая зона;
- `last_intent`, `intent_history` — история намерений, используется для детекции повторяющихся организационных вопросов и для аналитики;
- `context_scope` (`course_id`, `lesson_id`, `page_id`) — позиция студента в контентной иерархии;
- `last_widget_version` — версия последней проекции; используется фронтендом для защиты от устаревших обновлений;
- `learning_state` — опциональная подструктура, заполняемая только на LEARNING-маршруте: `topic`, `attempts`, `mastery_score`, `consecutive_wrong`, `last_strategy`, `explanation_used`, `misconceptions`.

Разделение общего контекста сессии и `learning_state` принципиально: организационные и навигационные маршруты не загрязняют тьюторские счётчики, а тьюторский маршрут сохраняет полный контекст между ходами.

Виджет проходит через три канонические сущности, отражающие уровни ответственности:

- 1) **WidgetIntent** — серверное намерение создать виджет под конкретный use-case. Содержит тип виджета, идентификатор плагина и контекст (зона, тема, уровень взаимодействия). Не содержит ни текста, ни DivKit JSON.
- 2) **WidgetDraft** — типизированный контентный payload, порождённый правилами или LLM. Содержит все тексты, ссылки, источники, предполагаемый набор действий. Это ещё не UI: DivKit-представление здесь отсутствует.
- 3) **WidgetProjection** — финальный виджет с заполненным DivKit JSON, зафиксированным набором `allowedActions`, размером (S/M/L/span-2/tall) и TTL. Только WidgetProjection уходит в ответ BFF.

Преобразование WidgetDraft → WidgetProjection выполняется двумя компонентами последовательно: Policy Gate валидирует схему черновика, проверяет допустимость типа для текущей зоны, допустимость всех `actionId`, размер контента и TTL; затем Projection Builder собирает итоговый DivKit JSON, добавляет Bento-подсказки размера и формирует метаданные (категорию, приоритет, калибруемость).

Каждый виджет несёт дополнительные три метаданных поля, используемых ранжированием и политиками:

- **category** — таксономия (`tutoring`, `operational`, `progress`, `commerce`, `profile`, `onboarding`); управляет подбором по зоне;
- **criticality** (`urgent/high/normal/low`) — приоритет размещения в Bento-сетке и скорость TTL;
- **calibrability** (`tenant/learner`) — может ли арендатор или сам студент переопределить контент или видимость.

Взаимодействие Smart Line описывается тремя основными потоками: *render*, *interaction*, *voice*.

Render-поток (первичное обращение студента):

- 1) Студент отправляет prompt и идентификатор активной зоны через Smart Line UI.
- 2) BFF извлекает из JWT контекст арендатора и студента, добавляет Request-Id и Correlation-Id, вызывает BC-16.
- 3) BC-16 загружает или инициализирует SmartLineSessionState из Redis.
- 4) Intent Router выбирает путь в зависимости от намерения (ORGANIZATIONAL, LEARNING_UNDERSTANDING, NAVIGATION_DISCOVERY, PROGRESS_REFLECTION, ACCOUNT_PROFILE, UNSUPPORTED).
- 5) BC-16 находит допустимые возможности плагинов для текущей зоны.
- 6) При необходимости выполняется запрос типизированного WidgetDraft через LLM Gateway.
- 7) Policy Gate валидирует черновик (схема, политика арендатора, допустимый тип виджета, размер, TTL).
- 8) Projection Builder собирает DivKit JSON и итоговый WidgetProjection.
- 9) BC-16 пишет запись аудита, публикует CloudEvents, сохраняет кэш проекции, возвращает список виджетов в BFF.
- 10) Фронтенд вызывает setData() DivKit-рендера и обновляет Widget Lake.

Interaction-поток (действие студента над виджетом): DivKit порождает custom_action, которое через BFF попадает в /interaction-эндпоинт BC-16. Сервер валидирует действие по реестру возможностей, обновляет состояние сессии, оценивает ответ (для LEARNING), возможно регенерирует Widget Lake, фиксирует аудит и публикует CloudEvents. При L3 ответ — обновление того же виджета; при L4 — новая версия Widget Lake.

Voice-поток. Голосовой ввод выполняется браузерным Web Speech API: промежуточные транскрипты показываются inline, финальный транскрипт отправляется как обычный prompt с отметкой inputMode=voice и объектом voiceMeta (уверенность распознавания, язык, длительность). Аудио не покидает браузер. Серверный контур обрабатывает голосовой запрос идентично текстовому. Обязательное требование доступности (WCAG 2.1 AA, error recovery): финальный транскрипт всегда редактируется студентом вручную перед отправкой — автоотправка запрещена.

Ядро тьюторского маршрута — семь агентов, перенесённых из исследовательского ноутбука и оформленных в Python-сервис adapstory-ai-smart-line-orchestrator. Каждый агент инкапсулирует ровно один промпт-шаблон и один тип операции над LLM; разделение состава агентов приведено в таблице 3.3.

Таблица 3.3 — Семь агентов тьюторского конвейера

Класс	Назначение	T	Выход
QuestionType-Classifier	маршрутизация вопроса: <code>organizational</code> , <code>learning</code> , <code>other</code>	0,0	str
OrganizationalAgent	ответ на организационные вопросы строго по RAG, без привлечения общих знаний LLM	0,1	текст
PlannerAgent	выбор стратегии: <code>socratic</code> , <code>hint</code> , <code>explain</code> , <code>verify</code> , <code>mastered</code>	0,1	JSON
TutorAgent	генерация реплики тьютора согласно выбранной стратегии	0,3	текст
IntentClassifier	per-message интент: <code>answer</code> , <code>chat</code> , <code>question</code> , <code>off_topic</code>	0,0	str
ChatAgent	обработка разговорных реплик и уточняющих вопросов	0,3–0,4	текст
EvaluatorAgent	оценка ответа: <code>correct</code> , <code>partial</code> , <code>score</code> , <code>misconception</code> , <code>feedback</code>	0,0	dict

Низкая температура ($T = 0$) применяется там, где ожидается структурированный JSON-выход и требуется воспроизводимость: классификация типа вопроса, классификация интента, оценка ответа. Повышенная температура ($T = 0,3–0,4$) используется для реплик тьютора и для ChatAgent, где нужна лингвистическая вариативность.

Детерминированные переходы. При всей генеративной природе LLM поведение конвейера сохраняется предсказуемым за счёт жёстких переходов:

– после `explain` следующая стратегия принудительно равна `verify` — студенту всегда задаётся короткий

проверочный вопрос после развёрнутого объяснения;

- при двух последовательных репликах интента chat тьютор принудительно переходит к explain — это предотвращает «зависание» в светской беседе;
- при `correct=True` или `score ≥ 0,85` сессия завершается терминальной карточкой `<< >>`;
- при `attempts ≥ 6` и стратегии отличной от `verify` сессия завершается карточкой `<< >>`, чтобы исключить бесконечный цикл.

Продакшн-маршрут работает поверх LangGraph: каждый агент — узел StateGraph, каждая переходная стрелка — условный рёберный предикат, поддерживающий контрольные точки, `retry` и `human-in-the-loop`. Это обеспечивает воспроизводимость и трассируемость всей тьюторской сессии через LangSmith.

Прямое обращение доменных сервисов к SDK провайдеров (OpenAI, YandexGPT, локальный Ollama) запрещено. Все LLM-вызовы проксируются через BC-10 LLM Gateway, выполняющий пять задач:

- 1) **Модель-агностичный интерфейс.** Унифицированный DTO `LlmRequest` с полями `model`, `messages`, `temperature`, `max_tokens`, `tools`. Выбор конкретного провайдера выполняется по полю `model` и политике маршрутизации арендатора.
- 2) **Маскирование персональных данных.** Перед отправкой в любую модель весь поток сообщений проходит через `PiiMasker`: e-mail, номер телефона, имя и фамилия студента заменяются псевдонимами `[EMAIL]`, `[PHONE]`, `[NAME]`. Обратное отображение применяется к ответу. Исключений не предусмотрено, даже для локальных Ollama-моделей.
- 3) **Учёт токенов на уровне арендатора.** Каждый вызов атрибутируется арендатору; Gateway поддерживает скользящее окно потребления и отклоняет запросы при превышении квоты.
- 4) **Паттерны устойчивости.** Circuit breaker (`fail_max=5`, `reset_timeout=60`), тайм-ауты (до 120 с для долгих LLM-вызовов), `retry` с экспоненциальной задержкой для идиоматичных операций.
- 5) **Версионирование промптов и guardrails.** Шаблоны хранятся в реестре с семантической версией и SHA-256 отпечатком; изменение шаблона требует новой версии. Guardrails валидируются минимум на 10 положительных и 10 негативных примерах в CI; превышение порога ложноположительных срабатываний 5 % считается блокирующим дефектом.

Все сквозные события Smart Line публикуются в Kafka в формате CloudEvents 1.0. Сборка конверта выполняется исключительно через разделяемый компонент `CloudEventEnvelopeConverter` — ручное построение JSON запрещено. Обязательные заголовки каждого сообщения: `X-Request-Id`, `X-Correlation-Id`, `X-User-Id`, `X-Tenant-Id`. Наименование типа события следует схеме `adapstory.{domain}.{entity}.{action}`; в типе обязателен суффикс версии `v{N}`.

Четыре типа событий, публикуемых тьютором:

- `adapstory.widget.rendered.v1` — публикуется после успешного рендера виджета; содержит `widget_id`, `session_id`, `learner_id`, `tenant_id`, имя плагина, стратегию, оценку ранжирования, TTL, SHA-256 отпечаток DivKit JSON, а также блок `cost` с числом использованных токенов, именем модели и оценкой USD-стоимости вызова;
- `adapstory.student.answered.v1` — отправляется после каждого ответа студента; несёт текст ответа, выход `EvaluatorAgent`, обновлённые счётчики;
- `adapstory.student.mastered.v1` — отправляется при переходе студента в состояние освоения темы; подписчики — `Behavioural Analytics`, граф компетенций;
- `adapstory.session.terminated.v1` — закрытие сессии по причине `mastered`, `max_attempts_reached`, `inactive_timeout`.

Потребители событий идиоматичны: каждая обработка сверяется с `ProcessedEventStore` по `event_id`. Kafka гарантирует `at-least-once`; упорядоченность на уровне сессии обеспечивается ключом партиционирования (по умолчанию `tenantId`, для строгого порядка в тьюторской сессии — `sessionId`).

Retrieval-слой в продакшене использует гибридную архитектуру. Векторный индекс тьютора размещается в Neo4j 5.x: узлы `LessonChunk` содержат эмбединг текста, а связи `TAUGHT_BY` → `Skill`, `PREREQUISITE_OF` и т. п. соединяют чанки с графом компетенций. Такая топология позволяет ранжировать найденные чанки не только по косинусной близости, но и по текущему положению студента в графе навыков.

Каждый Cypher-запрос обязательно фильтруется по `tenant_id`; это предотвращает перекрёстные запросы между арендаторами на уровне базы, а не только на уровне API. Лейблы узлов именуются в PascalCase (Lesson, Skill, Competency), отношения — в UPPER_SNAKE_CASE (COVERS_SKILL, PREREQUISITE_OF), свойства — в camelCase (chunkText, pageNumber, sourceName).

Извлечённый контекст передаётся в TutorAgent и OrganizationalAgent по шаблону, где каждый чанк сопровождается явными ссылками CHUNK_ID, SOURCE, PAGE. Это позволяет ответу ExplanationWidget содержать блок с прямыми отсылками на страницу и файл исходного материала.

Каждый запрос к Smart Line проходит цепочку фильтров: проверка подписи JWT, извлечение `tenant_id` и `learner_id`, валидация заголовка идиempотентности Idempotency-Key, тегирование текущего спана OpenTelemetry идентификатором арендатора. Все обращения к PostgreSQL, Neo4j, Redis и Qdrant параметризуются `tenant_id`, что исключает случайную утечку данных между арендаторами на уровне СУБД.

Аудит хранится в PostgreSQL (схема `bc_16_`) и содержит записи о каждом рендере виджета, каждом действии студента, каждом вызове LLM и его стоимости. Срок хранения регулируется политикой арендатора. Для диагностики применяется трассировка LangSmith с ролевым доступом `ai.tracer` — запись полных промптов и ответов LLM при уровне логирования INFO запрещена.

Телеметрия строится на трёх столпах:

- **OpenTelemetry (OTLP)** — единый протокол экспорта метрик, логов и трейсов. Каждый HTTP-запрос и каждая Kafka-обработка порождает корневой span; дочерние спаны соответствуют вызовам LLM Gateway, операциям с БД и шагам LangGraph.
- **Структурированное логирование в JSON** (structlog для Python, Logback с encoder в JSON для Java). Сообщения пишутся исключительно на английском. Обязательные поля: `correlation_id`, `tenant_id`, `user_id`, `event`.
- **Метрики SLO/SLI** в Prometheus: p_{50} / p_{95} / p_{99} задержки `render-эндпоинта`, RPS, уровень ошибок, потребление токенов, уровень освоения по темам. Grafana-дашборды отражают бюджет ошибок и прогноз его выгорания.

Обязательны три пробы Kubernetes для каждого сервиса: `livenessProbe` (всегда 200), `readinessProbe` (проверка Redis, Kafka, Neo4j, LLM Gateway), `startupProbe` (ожидание загрузки локальных моделей).

Стенд разработки работает на K3s с локальным Ollama для GPU-LLM. Продакшн-развёртывание выполняется строго по модели GitOps: манифесты хранятся в отдельном репозитории, ArgoCD синхронизирует состояние кластера с Git. Ручные операции `kubectl apply/edit/patch` против продакшн-кластера запрещены.

Образы контейнеров подписываются через Cosign, SBOM публикуется вместе с артефактом; неподписанные образы в продакшн не допускаются. Тег образа — `{version}-{git-sha}`; `latest` запрещён.

Для каждого сервиса обязательны HPA (целевая загрузка CPU 70 %, минимум 2 реплики в продакшене) и PodDisruptionBudget (минимум 1 всегда доступная реплика). Graceful shutdown по SIGTERM в течение 30 секунд; `terminationGracePeriodSeconds` Kubernetes — не менее 35 секунд.

3.2 Разработка и имплементация решения MVP

Задача MVP-реализации сформулирована как перенос функциональной логики исследовательского ноутбука в работающий веб-сервис с минимально достаточным фронтендом при сохранении полной семантики семи агентов и интерактивной тьюторской сессии. MVP сознательно избегает внедрения инфраструктурных компонентов целевой архитектуры (LLM Gateway, Kafka, Neo4j-векторный индекс, GitOps, многоарендность, маскирование РИ), оставляя их на этап промышленной реализации.

Объём MVP определяется таблицей включений и исключений (таблица 3.4).

Такое разграничение позволило за счёт отсутствия инфраструктурных обвязок провести UX-тестирование полного цикла «вопрос студента → проекция виджетов → ответ студента → следующий виджет» при стоимости разработки менее 40 инженерных часов.

В состав MVP вошли:

- **Python 3.12+** как язык прикладного уровня; обоснование — совместимость со стеком LLM-обучения,

Таблица 3.4 — Границы MVP-прототипа тьютора

Включено в MVP	Отложено до продакшн-версии
FastAPI-сервис с тремя HTTP-эндпоинтами	LLM Gateway (BC-10) как отдельный слой
Перенос всех 7 агентов без изменений семантики	Kafka / CloudEvents и подписчики аналитики
RAG через Qdrant на готовой коллекции курса	Neo4j-векторный индекс и граф компетенций
Пять фабрик виджетов, совпадающих по схеме с Widget Lake	Policy Gate + Projection Builder как отдельные компоненты
Статический HTML/CSS/JS-фронтенд с голосовым вводом	Интеграция с рендерером DivKit в продакшн-фронтенде
Автоматический fallback между real- и mock-бэкендом	Реестр плагинов и capability discovery
Детерминированные переходы стратегий тьютора	Многоарендность, JWT, RBAC, квоты арендатора
Smoke-тесты через curl и браузер	PII-маскирование, structlog, OTLP-экспорт
In-memory хранение сессии	Redis-кэш состояния StudentStateCache

наличие готовых SDK (OpenAI, Qdrant-client) и развитая типизация;

- **FastAPI 0.136** — асинхронный веб-фреймворк с автоматической генерацией OpenAPI, валидацией через Pydantic и нативной поддержкой lifespan-контекстов;
- **uvicorn 0.44** с параметрами `host=127.0.0.1`, `port=5173`, `log_level=info`; локальная привязка исключает экспозицию в LAN на этапе прототипирования;
- **openai 2.32** — официальный SDK для вызовов `chat.completions` и `embeddings`;
- **qdrant-client 1.17** с методом `query_points` (в версиях ≥ 1.10 заменил устаревший `search`);
- **pydantic 2.13** — валидация request/response DTO; поле `alias` обеспечивает совместимость «camelCase на входе — snake_case внутри»;
- **ванильный JavaScript** — отсутствие фронтенд-фреймворка осознанно, поскольку цель прототипа — проверить UX, а не архитектуру UI-слоя.

Зависимости зафиксированы в файле `requirements.txt` в виде нижних границ версий; полный lock-файл не поставляется, так как прототип не развёртывается в CI/CD.

Прототип размещён в директории `smart-line-prototype/` и имеет плоскую структуру:

```
smart-line-prototype/
  index.html      -
  styles.css      -      Adapstory Design System
  app.js          -      : mock-      + RealBackend
  README.md       -
  backend/
    server.py      - FastAPI-      (1065 LOC)
    requirements.txt
    .env           -      (git-ignored)
    .env.example  -
```

Фронтенд и бэкенд сознательно разделены на уровне каталогов, но объединены при запуске: FastAPI монтирует родительскую директорию как статический источник (`app.mount("/", StaticFiles(...))`). Это позволяет обращаться по единому origin `http://127.0.0.1:5173/` как к интерфейсу, так и к API-эндпоинтам — CORS-политика упрощается, а прототип развёртывается одной командой.

Все семь агентов перенесены из ноутбука как независимые классы (см. таблицу 3.3). Каждый класс инкапсулирует один промпт-шаблон на русском языке и один вызов `ask_llm` — фасадной функции над `openai.chat.completions.create` с параметрами `model=CHAT_MODEL`, `max_tokens=700`. Выбор модели

задаётся переменной окружения CHAT_MODEL (по умолчанию gpt-4o-mini).

Алгоритм обработки одного хода тьютора иллюстрируется следующей блок-схемой (рисунок 3.2).

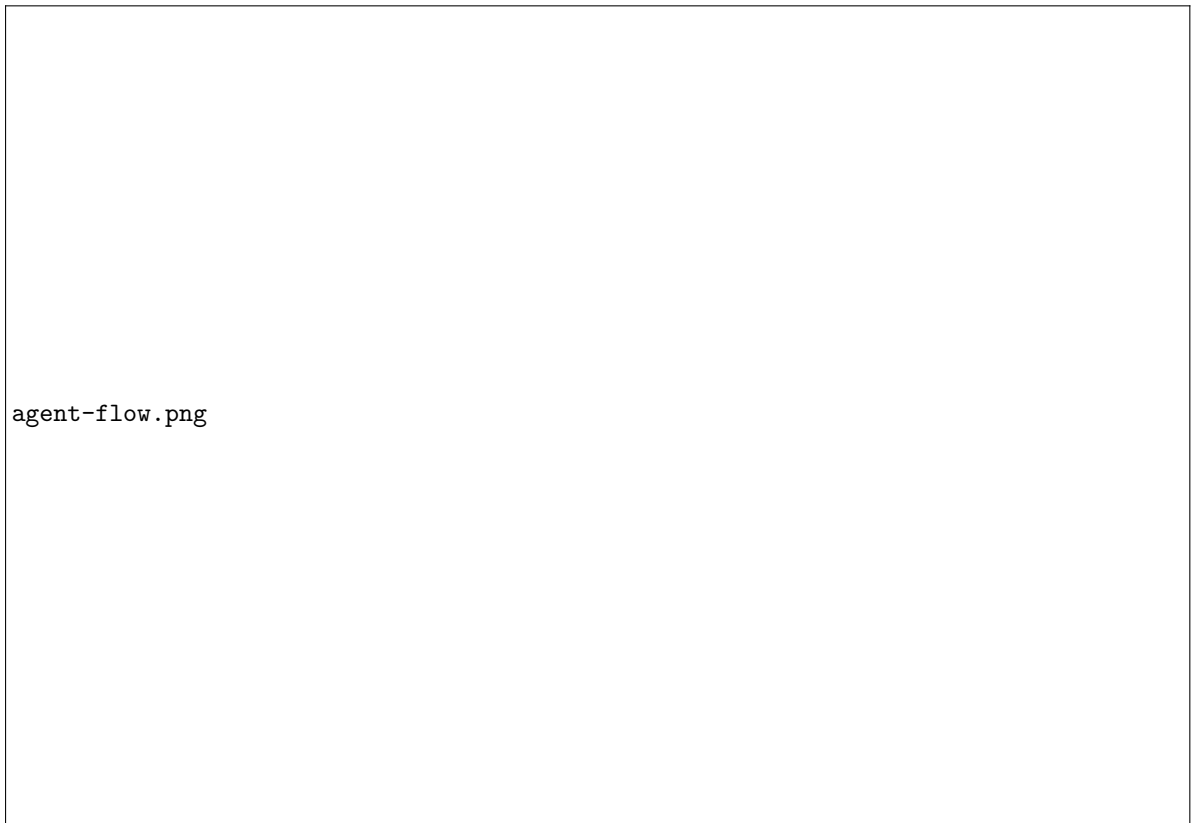


Рисунок 3.2 — Поток обработки хода тьютора (семь агентов)

Общий контур обработки одного шага тьюторского маршрута:

- 1) `QuestionTypeClassifier` определяет тип входящего вопроса студента.
- 2) Если тип — `organizational`, `OrganizationalAgent` формирует ответ по RAG и возвращает единственный виджет `SmartLineAnswerCard`.
- 3) Если тип — `other`, возвращается виджет `InfoCard` с сообщением о несоответствии теме курса.
- 4) Если тип — `learning`:
 - 1) выполняется RAG-поиск (функция `retrieve_context`, $k = 10$);
 - 2) `PlannerAgent` выбирает стратегию на основе текущего `StudentState`;
 - 3) `TutorAgent` порождает реплику согласно стратегии;
 - 4) результат упаковывается в `AskUserQuestion` или `ExplanationWidget` в зависимости от стратегии; дополнительно возвращается `SkillProgress`.
- 5) На ответ студента (POST /api/interact):
 - 1) `IntentClassifier` определяет интент (`answer`, `chat`, `question`, `off_topic`);
 - 2) при `chat` обрабатывает `ChatAgent`; если это уже вторая подряд реплика типа `chat`, принудительно переходит к `explain`;
 - 3) при `answer` `EvaluatorAgent` оценивает ответ, `PlannerAgent` выбирает следующую стратегию;
 - 4) при `question` `ChatAgent` отвечает по RAG и возвращает тьютора к прежнему вопросу;
 - 5) при `off_topic` возвращается предыдущая реплика с напоминанием о теме.

Функция `retrieve_context(question, top_k=10)` реализует семантический поиск по векторной базе:

- 1) `embed_query` порождает эмбединг запроса с использованием модели `text-embedding-3-large` (3072 измерения);
- 2) вызов `QdrantClient.query_points(collection_name= "presentations_industrix_openai", query=vector, limit=10, with_payload=True)` возвращает список ближайших точек;

- 3) из полезной нагрузки извлекаются поля `text`, `page`, `source`; пустые чанки отбрасываются;
- 4) результат кэшируется в `StudentState.contexts` и переиспользуется в последующих шагах сессии — это исключает повторные запросы к Qdrant при неизменной теме и ускоряет p_{95} до ~ 2 сек.
- Функция `build_context` конкатенирует выбранные чанки в единый текстовый блок, снабжая каждый идентификатором, источником и страницей в формате

```
CHUNK_ID: 0
SOURCE: syllabus.pdf
PAGE: 3
```

TEXT:

```
...
---
```

```
CHUNK_ID: 1
...
```

Такая форма необходима, чтобы TutorAgent при стратегии `explain` мог процитировать источник в `ExplanationWidget` с указанием страницы.

Состояние сессии описывается датаклассом `StudentState` с явно перечисленными полями: `session_id`, `topic`, `attempts`, `mastery_score`, `history`, `misconceptions`, `last_strategy`, `explanation_used`, `consecutive_wrong`, `consecutive_chat`, `contexts`, `last_tutor_msg`, `active_widget_id`, `force_next_strategy`. Список сессий хранится в словаре `SESSIONS: Dict[str, StudentState]` в памяти процесса.

Перенос ноутбука сохранил все инварианты переходов без изменений:

- после стратегии `explain` следующая стратегия принудительно равна `verify` (через установку `force_next_strategy="verify"`);
- при двух последовательных репликах типа `chat` тьютор принудительно переходит к `explain`; счётчик `consecutive_chat` сбрасывается;
- при `correct=True` или `score $\geq$ 0,85` сессия завершается терминальной карточкой `<< >>`;
- при `attempts $\geq$ 6` и стратегии отличной от `verify` сессия завершается карточкой `<< >>`.

Для каждой сессии накапливается диалоговая история в формате " : ... " / " : ...", обрезаемая до последних 6 сообщений при передаче в `ChatAgent` (во избежание переполнения контекстного окна и неявного ухудшения качества поздних реплик).

Выход каждого агента проецируется не в сырой текст, а в декларативный JSON-виджет. В MVP реализованы пять фабрик виджетов (таблица 3.5).

Таблица 3.5 — Фабрики виджетов MVP-прототипа

Фабрика	Тип виджета	IL	Сценарий использования
<code>make_ask_user_question</code>	<code>AskUserQuestion</code>	L3	сократический вопрос, подсказка, <code>verify</code> , <code>chat-reminder</code>
<code>make_explanation_widget</code>	<code>ExplanationWidget</code>	L1	развёрнутое объяснение со ссылками на источники
<code>make_answer_card</code>	<code>SmartLineAnswerCard</code>	L1	ответ на организационный вопрос / <code>fallback</code>
<code>make_skill_progress</code>	<code>SkillProgress</code>	L1	индикатор прогресса, всегда сопровождает <code>learning</code>
<code>make_info_card</code>	<code>InfoCard</code>	L1	дефолты зон, терминальные карточки

Каждый виджет содержит обязательный конверт: `widgetId` (UUID-фрагмент), `category`, `criticality`, `calibrability`, объект `layout` (`size`, `cornerRadiusPreset`, `priority`), `interactionLevel`, `allowedActions`, а также типозависимый `payload`. Такая форма совпадает с продакшн-контрактом `Widget Lake`, что делает MVP-прототип совместимым с промышленным рендерером интерфейса без изменений

схемы. В частности, поле `allowedActions` для `AskUserQuestion` содержит единственное значение `smart_line:submit_answer` — это соответствует реестру допустимых действий BC-16 Policy Gate.

Сервис предоставляет три HTTP-эндпоинта и статический фронтенд:

- 1) **GET /api/health** — проба живости и эхо конфигурации. Ответ: `{ok, openai, qdrant, collection, chat_model, embed_model, sessions}`. Используется фронтендом для автоматического выбора режима.
- 2) **POST /api/render** — начальный шаг сессии. Запрос: `{prompt, space, sessionId?, inputMode}`; ответ: `{sessionId, intent, widgets[], state, strategy?}`. Для значений `space ≠ LEARNING` возвращаются детерминированные карточки без обращения к LLM.
- 3) **POST /api/interact** — обработка действия студента над виджетом. Запрос: `{sessionId, widgetId?, actionId, input}`; ответ аналогичен `/api/render` и дополнительно содержит `messageIntent`, `nextStrategy`, `evaluation`, опциональный `sessionStatus`. Поддерживается единственный `actionId` `smart_line:submit_answer`; иные значения приводят к HTTP 400. Отсутствие сессии — HTTP 404.
- 4) **GET /** — статический монтинг директории `smart-line-prototype/`. Выполняется после маршрутов API, что исключает конфликт префиксов.

Обработчик всех исключений возвращает HTTP 500 с телом `{error, type}`. Отсутствие `OPENAI_API_KEY` приводит к HTTP 503 при первом обращении к LLM (ленивая инициализация клиента) — это явно указывает на проблему конфигурации, а не на отказ сервиса.

Исходная версия `app.js` представляла собой автономный клиентский мок без обращений к бэкенду. В рамках MVP клиент доработан минимально инвазивно: добавлены два новых модуля — `RealBackend` и `probeBackend` — а также визуальный индикатор текущего режима.

На загрузке страницы выполняется запрос `GET /api/health`. Логика выбора режима описывается таблицей 3.6.

Таблица 3.6 — Условия выбора режима фронтенда

Условие на момент загрузки	Режим	UX-эффект
<code>/api/health</code> недоступен (таймаут, сетевая ошибка)	mock	полный in-browser тьютор
<code>/api/health = 200, openai=false</code>	mock	тот же; исключается полу-реальный режим
<code>/api/health = 200, openai=true</code>	real	каждый ход — сетевой вызов к FastAPI

Визуальный маркер режима — пилюля в шапке страницы: зелёная надпись `<<real>>` с именем модели или янтарная `<<mock>>`. Приветственные карточки различаются по содержанию: для `real`-режима они указывают имя модели, имя коллекции RAG и ожидаемую задержку (2–5 сек на ход); для `mock`-режима — инструкцию по запуску бэкенда.

Ошибки сетевых вызовов перехватываются и отображаются в виде toast-уведомлений в правом верхнем углу; стек-трейсы пишутся в консоль браузера. Потеря сессии (например, после перезапуска бэкенда) выражается в HTTP 404 на следующий `/api/interact`; пользователь видит toast, после чего отправка нового запроса в строке ввода создаёт свежую сессию.

MVP реализует требование голосового ввода через браузерный API `window.SpeechRecognition` (или `webkitSpeechRecognition`) в соответствии с v1.2-контрактом продакшн-архитектуры. Распознавание запускается по кнопке микрофона; промежуточные транскрипты выводятся как toast-уведомления. Финальный транскрипт подставляется в поле ввода, но не отправляется автоматически — студент обязан просмотреть и подтвердить текст (требование доступности WCAG 2.1 AA по восстановлению после ошибок).

Метаданные распознавания (уверенность, язык, длительность) сохраняются в `dataset` элемента ввода и передаются в запрос как `voiceMeta`. При отсутствии поддержки Web Speech API кнопка микрофона скрывается автоматически, а подсказка рекомендует использовать Chrome или Edge.

Для проверки работоспособности MVP предусмотрен воспроизводимый набор smoke-тестов, который выполняется в Windows 11 git-bash:

- 1) запуск сервиса — `python server.py` в директории `backend/` при заданном `OPENAI_API_KEY`; корректный старт фиксируется сообщением `uvicorn running on http://127.0.0.1:5173`;
- 2) проверка живости — `curl http://127.0.0.1:5173/api/health` должен вернуть объект с `ok=true` и корректными именами моделей;
- 3) проверка маршрута `LEARNING` — `POST /api/render cprompt="unit economics", space="LEARNING"`; ожидаемый ответ — два виджета (`AskUserQuestion`, `SkillProgress`) и стратегия `socratic`;
- 4) проверка фронтенда — `http://127.0.0.1:5173/` в браузере; убедиться, что в шапке страницы видна зелёная пиллюля `<<real>>` с именем модели, а приветственный блок содержит карточку о подключении к OpenAI и Qdrant.

Все четыре шага успешно прошли на стенде разработчика (Windows 11, Python 3.12, uvicorn 0.44, OpenAI gpt-4o-mini, Qdrant presentations_industrixx_openai).

Набор осознанных ограничений MVP сведён в таблицу 3.7 с явным указанием пути устранения каждого из них в продакшн-реализации.

Таблица 3.7 — Ограничения MVP и план переноса в продакшн

#	Ограничение MVP	Путь в продакшн
L1	Состояние сессии в памяти процесса; теряется при рестарте	Redis + StudentStateCache (BC-16), TTL 1800 с с автопродлением
L2	Прямые обращения к OpenAI SDK без шлюза	Маршрутизация через BC-10 LLM Gateway; таймауты, circuit breaker, квоты арендатора
L3	PII не маскируется перед LLM	PiiMasker из adapstory_commons.privacy как обязательный слой Gateway
L4	CORS открыт на *, bind на 127.0.0.1	Ingress-контроллер + политика CORS по арендатору
L5	Стандартный logging вместо JSON	structlog в JSON + OTLP-экспорт
L6	Отсутствие проб liveliness/readiness/startup	Middleware из adapstory-api-commons
L7	Тестовое покрытие отсутствует	pytest + pytest-asyncio, покрытие $\geq 65\%$
L8	Нет типовой проверки	mypy --strict в CI-пайплайне
L9	Отсутствие эмиссии событий Kafka	4 события CloudEvents 1.0 по контракту BC-16
L10	Qdrant вместо Neo4j-векторного индекса	Миграция на Neo4j 5.x с графом компетенций, CDC PostgreSQL → Neo4j
L11	Отсутствие Policy Gate и Projection Builder	Валидация черновиков перед DivKit-проекцией на стороне BC-16
L12	Нет реестра плагинов	BC-02 Plugin Lifecycle, capability discovery, версионирование шаблонов

Каждое ограничение напрямую отображается в один или несколько пунктов целевой архитектуры (раздел 3.1), что обеспечивает преемственность между MVP и промышленной версией: семь агентов, пять фабрик виджетов, детерминированные переходы стратегий и контракт HTTP-API сохраняются как есть; к ним добавляются инфраструктурные компоненты (шина событий, кэш, шлюз моделей, многоарендность, наблюдаемость).

В ходе реализации MVP продемонстрирована сквозная работоспособность предложенной тьюторской архитектуры. Исследовательский ноутбук успешно перенесён в FastAPI-сервис объёмом 1065 строк кода; все семь агентов (`QuestionTypeClassifier`, `OrganizationalAgent`, `PlannerAgent`, `TutorAgent`, `IntentClassifier`, `ChatAgent`, `EvaluatorAgent`) сохранили исходную семантику, RAG-интеграция через Qdrant подтверждена, проекция виджетов Widget Lake согласована с продакшн-контрактом, детерминированные переходы стратегий (`explain` → `verify`, `chat` × 2 → `explain`, `mastered`, `max_attempts_reached`) воспроизведены без модификации.

Минимально инвазивная доработка фронтенда обеспечила автоматический выбор между реальным и мок-бэкендом, что позволяет демонстрировать UX-сценарий как с использованием LLM, так и в автономном режиме. Smoke-тестирование в режиме `real` успешно прошло со всеми четырьмя проверочными шагами, подтвердив корректность HTTP-контракта и интеграции с OpenAI и Qdrant.

Сопоставление набора из двенадцати ограничений MVP с двенадцатью компонентами целевой архитектуры

(таблица 3.7, раздел 3.1) даёт непротиворечивый план переноса в продакшн без переписывания доменной логики агентов. Это подтверждает гипотезу о том, что предложенная архитектура (продуктовая — H1, техническая — H2, бизнесовая — H3) реализуема в виде промышленного продукта, а текущий MVP представляет собой её функционально замкнутое, но инфраструктурно упрощённое воплощение.

3.3 Продуктовые направления развития

Текущий MVP завершает тьюторскую сессию по достижении мастерства или исчерпанию лимита попыток; каждая следующая сессия начинается с пустого `StudentState`. Развитие этого контура ведёт к *непрерывному наставнику*: долговременной памяти студента, накапливающей историю освоенных тем, обнаруженных заблуждений, предпочитаемых способов объяснения и темпа обучения. Техническая реализация опирается на граф знаний в Neo4j, где узлы — компетенции, а рёбра — зависимости освоения; поверх графа ведётся отдельное профилирование каждого студента. Такой подход изменяет характер взаимодействия с «эпизодическое» на «сквозное»: тьютор предлагает релевантные темы исходя из пробелов в графе, а не только по прямому запросу студента.

MVP поддерживает четыре стратегии (`socratic`, `hint`, `explain`, `verify`) плюс терминальное состояние `mastered`. Перспективные направления расширения:

- **Worked-example pedagogy** — пошаговый разбор типового примера с последующим предложением студенту решить аналогичный самостоятельно; особенно эффективен в точных дисциплинах;
- **Elaborative interrogation** — стратегия, требующая от студента объяснить, *почему* нечто верно, а не только *что* верно; подтверждено в исследованиях памяти;
- **Interleaved practice** — чередование задач из разных тем внутри одной сессии; требует графа компетенций для подбора смежных задач;
- **Spaced retrieval** — отложенные возвращения к теме через интервалы по закону забывания Эббингауза; реализуется планировщиком на уровне непрерывного наставника;
- **Error-driven dialogue** — целенаправленная индукция типовых заблуждений с их последующим разбором; требует библиотеки диагностических заданий.

Каждая новая стратегия оформляется как новый промпт-шаблон `TutorAgent` и соответствующее условие в `PlannerAgent`. Добавление стратегии не требует переписывания оркестрации — это прямое следствие принятой в главе 3 плагиновой декомпозиции.

Текущий тьютор обращается ко всем студентам одинаково. Введение *когнитивного профиля* позволяет адаптировать стиль общения, длину объяснений, плотность источников и скорость перехода между стратегиями. Профиль состоит из измерений:

- *preferred explanation depth* — tl;dr vs. детальное;
- *example preference* — абстрактные определения vs. конкретные примеры;
- *pace* — быстрый цикл «вопрос–ответ» vs. медленный разбор;
- *language register* — академический vs. разговорный;
- *feedback sensitivity* — прямая корректура vs. мягкая наводка.

Профиль формируется за счёт явных настроек студента, неявной телеметрии (длительность чтения объяснения, число обращений к подсказке) и модели коллаборативной фильтрации по похожим студентам в пределах арендатора.

MVP работает только с текстом. Для повышения качества тьюторинга предусмотрены три направления расширения модальностей:

- 1) **Text-to-image-объяснения**. При стратегии `explain` тьютор порождает не только текст, но и иллюстрацию (схему, сравнительную таблицу, Mind Map) через модель уровня DALL-E 3 или FLUX. Иллюстрация упаковывается в новый тип виджета `ExplanationWidgetWithFigure`.
- 2) **Голосовой вывод (TTS)**. Для зон `LEARNING` в режиме `hands-free` `ExplanationWidget` дополнительно проигрывается синтезированным голосом (Silero для русского, ElevenLabs — премиум). Это расширяет доступность и поддерживает сценарии мобильного обучения в дороге.

3) **AI-подкасты.** По завершении модуля тьютор генерирует 5–7-минутный подкаст-диалог двух голосов, обобщающий ключевые идеи; концепция заимствована из NotebookLM и адаптирована под русскоязычный контент.

Текущий RAG в MVP использует только плотные эмбединги (`text-embedding-3-large`) и косинусную близость. Для образовательного контента этого недостаточно: многие ключевые термины курса редки и плохо ловятся плотными эмбедингами. Путь развития — *гибридный ретривал*:

- **BM25** поверх того же корпуса обеспечивает лексическое совпадение терминов;
- **Dense retrieval** отвечает за семантическое сходство;
- **Reciprocal Rank Fusion** объединяет оба ранжирования;
- **Cross-encoder re-ranker** (уровень BGE-reranker) поднимает топ-3 наиболее релевантных чанков.

Ожидаемый прирост *Ragas context precision* — 10–15 п. п. по сравнению с чисто плотным поиском; *faithfulness* ответа тьютора — 5–8 п. п.

Все промпты в MVP решают задачу через zero-shot prompting к `gpt-4o-mini`. Следующий уровень — supervised fine-tuning базовой модели (семейство Qwen 2.5 или LLaMA 3) на собственном образовательном корпусе.

Обучающие данные:

- транскрипты успешных сессий Customer Development;
- экспертно размеченные ответы с корректной методикой;
- диалоги с `gpt-4o-mini`, прошедшие ревью методистов (как «учительский» корпус).

Fine-tuning обещает три эффекта: снижение стоимости одного хода (меньшая модель справляется с задачей базовой модели), повышение культурной и языковой адекватности (русскоязычный образовательный контекст), более строгая выдержка формата JSON-выхода.

Для образовательных организаций с повышенными требованиями к обработке персональных данных предлагается конфигурация *on-premise* с локальной LLM (Ollama, vLLM). Такая конфигурация требует:

- абстракции провайдера в LLM Gateway (архитектурно заложено, см. §3.1.10);
- серверов с GPU (минимум NVIDIA A10 24 GB для Qwen 2.5 7B);
- отдельного коннектора в Helm-чарте (`llm.provider=ollama`);
- расширения CI-стенда для автоматических тестов совместимости с локальной моделью.

Каждое изменение промпт-шаблона или переключение модели может ухудшить метрики. Для безопасного выката требуется встроенное A/B-тестирование:

- версионирование шаблонов с семантической версией и SHA-отпечатком (уже предусмотрено в архитектуре);
- деление трафика арендатора между вариантами (контрольный 80 % / экспериментальный 20 %);
- онлайн-метрики качества: *mastery rate*, *attempts per mastery*, *chat-to-answer ratio*, *student NPS*;
- автоматический rollback при деградации ключевых метрик на 10 %.

MVP выбирает RAG-чанки только по близости к запросу. Развитие — *кондиционирование поиска на графе компетенций*: из множества ретривнутых чанков приоритет получают те, что связаны с навыками, находящимися «на границе» зоны ближайшего развития студента (*sensu* Vygotsky). Техническая реализация: Cypher-запрос в Neo4j, который фильтрует чанки по связям `TAUGHT_BY`, `PREREQUISITE_OF` и `CURRENTLY_LEARNING`.

Платформа Adapstory предусматривает второй AI-сервис — методиста, порождающего структуру и содержимое курса. Прямое использование артефактов методиста в тьюторе даёт синергию: каждый урок, сгенерированный методистом, сопровождается уже размеченным банком проверочных вопросов, предвычисленным набором «типовых заблуждений» и готовыми промпт-контекстами для `ExplanationWidget`. Тьютор использует их без повторного похода в RAG при стартовом запросе по теме, что сокращает задержку первого виджета до ≤ 1 сек и снижает стоимость холодного старта сессии.

Текущие виджеты преимущественно работают на уровнях L1–L3. Развитие — массовое внедрение L4 (перегенерация всей Bento-сетки после действия студента) и L5 (запуск долгой сессии, например, «получасовая практика с ментором»). L4 даёт принципиально новый UX: `Widget Lake` превращается в живую поверхность, реагирующую на каждое действие целиком, а не только в части одного виджета. L5 превращает `Smart Line` из точки входа в полнофункциональный тьюторский зал, сравнимый по времени взаимодействия с живым

занятием.

Тьютор, действующий автономно, лишает преподавателя контроля над качеством обучения. Для снятия этого ограничения предусматривается преподавательский дашборд с тремя функциями: *просмотр трассы* сессий выбранного студента; *инъекция комментариев* в промпт тьютора для конкретной темы; *ручное одобрение* очередного шага в сложных сценариях (human-in-the-loop-режим, доступный в LangGraph по умолчанию). Преподаватель сохраняет владение методикой, одновременно используя LLM как «ускоритель».

MVP работает только в браузере Chrome/Edge на десктопе. Для студенческой базы, 60–70 % которой учится с мобильных устройств, требуется нативное приложение (iOS + Android) с поддержкой офлайн-режима. Офлайн-режим реализуется за счёт заранее заэкшированного набора виджетов Widget Lake и локальной легковесной модели (Qwen 2.5 0.5B на устройстве) для простых ответов. При восстановлении связи сессия синхронизируется с облачной копией.

Архитектура Smart Line изначально плагиновая: это позволяет в будущем открыть *маркетплейс* сторонних тьюторов-плагинов. Третьи стороны (частные методисты, издательства, корпоративные L&D-команды) могут поставлять тьюторские конвейеры для узких доменов: медицинская сертификация, сдача CFA, подготовка к ЕГЭ, внутреннее обучение compliance. Маркетплейс опирается на BC-02 Plugin Lifecycle: подпись артефактов, версионирование, изоляция промптов.

Текущие метрики (mastery_score, attempts, NPS) отражают краткосрочное качество одной сессии. Открытая исследовательская задача — *долгосрочная эффективность*: сохраняется ли освоенное знание через 4, 8, 26 недель; как это зависит от выбранной стратегии тьютора; как spaced retrieval влияет на retention-кривую. Протокол исследования: контрольная группа в B1 (ручной методист), экспериментальные группы с различными стратегиями SRS (Leitner, Anki-подобная, adaptive).

Применение LLM в образовании несовершеннолетних требует ужесточённых guardrails:

- защита от индукции самоповреждающего поведения;
- защита от ответов, навязывающих политическую или религиозную позицию;
- защита от переноса академической нечестности (генерация готового решения вместо наводящего);
- возрастная адаптация лексики и содержания.

Каждая политика оформляется как отдельный guardrail с независимым версионированием и CI-тестами.

Текущий EvaluatorAgent распознаёт заблуждение «ad-hoc» — в поле misconception JSON-ответа. Развитие — *типологизированный каталог заблуждений* по каждой теме (построенный экспертно или автоматически из массива ответов студентов с кластеризацией). Детекция становится классификационной задачей по известным классам, а не свободной генерацией, что повышает точность и позволяет адресно выбирать корректирующее объяснение.

Отдельное направление — *ранняя детекция выгорания студента*. Признаки: рост consecutive_chat, сокращение длины ответов, удлинение пауз, обращения к «пропустить». Агент-мониторинг мотивации (MotivationAgent) проактивно предлагает смену темы, отложить сессию или переключиться на more lightweight-формат. Это перекликается с исследованиями в области Affective Tutoring Systems.

Представленные направления развития неравнозначны по стоимости реализации и ожидаемому эффекту. Приоритизация сведена в таблицу 3.8, где каждое направление отнесено к одному из трёх горизонтов с учётом зависимостей от инфраструктурных предпосылок.

Приоритизация подчинена правилу: *сначала то, что использует уже развёрнутую инфраструктуру, но приносит продуктовую ценность*. Так, гибридный RAG и L4-виджеты в первые 6 месяцев не требуют новых внешних сервисов, но существенно улучшают качество обучения и UX. Во второй горизонт попадает тяжёлая интеграция (мобильный стек, локальные модели), в третий — долгосрочные научно-исследовательские программы.

Выделены четырнадцать направлений развития ИИ-тьютора, сгруппированных по четырём доменам: продуктовому, техническому, интеграционному, научно-исследовательскому. Каждое направление имеет явные технические предпосылки и отнесено к одному из трёх временных горизонтов — 6, 12 и 24 месяца. Совокупность направлений обеспечивает преемственность: на каждом горизонте тьютор развивается без отказа от уже принятой архитектуры, дополняя её новыми возможностями.

Таблица 3.8 — Дорожная карта развития ИИ-тьютора

Горизонт	Направление	Предпосылки / риск
6 мес.	гибридный RAG (BM25 + re-ranker)	есть корпус; риск — инфраструктура индексов
	Widget Lake L4 в LEARNING	есть Widget Lake; риск — UX-согласование
	A/B-тестирование промптов	требуется метрик — уже есть
	связка «методист → тьютор»	требуется запуск методиста в продакшн
12 мес.	мультимодальность (text-to-image, TTS)	стоимость иллюстраций, выбор провайдеров
	локальные Ollama-модели для арендаторов on-premise	требуется GPU-инфраструктуры
	преподавательский дашборд + human-in-the-loop	требуется расширения API BC-16
	граф компетенций и adaptive retrieval	требуется расширения Neo4j-схемы
24 мес.	fine-tuning базовой модели	требуется $\geq 10\,000$ размеченных диалогов
	мобильное приложение + офлайн-режим	требуется iOS/Android команды
	маркетплейс плагинов	юридический и compliance-контур
	долгосрочные retention-исследования	требуется ≥ 6 мес. лонгитюдального сбора

Принципиально важно, что ни одно из направлений не требует переписывания ядра: декомпозиция на ограниченные контексты, плагиновая природа возможностей и модель-агностичный LLM Gateway — инженерные решения, специально выбранные в главе 3 ради эволюционируемости. Это подтверждает, что Smart Line является не конечной точкой, а жизнеспособной платформой для итеративного развития ИИ-тьютора на горизонте нескольких лет.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы разработана бизнес-концепция и техническая реализация образовательной платформы нового поколения Adapstory с интегрированным интеллектуальным помощником обучения Smart Line, построенным на основе больших языковых моделей и многоагентной архитектуры Retrieval-Augmented Generation. Поставленная цель — разработка концепции AI LMS-платформы, реализующей интеллектуального тьютора для автоматизации образовательных процессов и повышения эффективности онлайн-обучения, — достигнута; выполнены все семь задач, сформулированных во введении. Работа выстроена по принципу последовательного перехода от анализа предметной области и пользовательских потребностей через экспериментальное исследование к промышленно ориентированной архитектурной концепции и работоспособному прототипу, что обеспечивает методологическую целостность и проверяемость каждого выдвинутого тезиса.

Методологически исследование объединило несколько взаимодополняющих подходов: количественный анализ рыночных данных и аналитических отчётов по сегменту EdTech, качественные глубинные интервью с преподавателями различных форматов обучения, методологию Jobs To Be Done для формализации пользовательских задач, сравнительный архитектурный анализ существующих AI-решений в образовании, экспериментальную оценку трёх архитектурных сценариев AI-тьютора, итеративное прототипирование с переносом исследовательской логики в работающий веб-сервис, а также проектирование промышленной архитектуры с учётом требований регламентов разработки, многоарендности и наблюдаемости. Такая комбинация методов позволила связать продуктовые, педагогические, инженерные и операционные аспекты в единую концепцию, не редуцируя ни один из них до второстепенного.

В части анализа рынка и технологий проведено комплексное исследование российского и мирового сегментов EdTech. Установлено, что российский рынок образовательных технологий демонстрирует устойчивый рост: совокупная выручка ста крупнейших компаний по итогам третьего квартала 2025 года составила около 42 млрд рублей при годовом приросте порядка 12 %. Мировой рынок искусственного интеллекта в образовании оценивается в 5,88 млрд долларов в 2024 году с прогнозом до 32 млрд долларов к 2030 году при среднегодовом темпе роста более 30 %; рынок AI-тьюторов — около 2 млрд долларов в 2025 году с прогнозом превышения 10 млрд долларов к 2032 году. Сравнительный анализ ведущих платформ (Skillbox, Skyeng, Яндекс Практикум, Учи.ру, GetCourse, Coursera, Duolingo, Khan Academy, Pearson, Brainly) показал, что несмотря на разнообразие функциональных возможностей, существующие решения преимущественно ориентированы на доставку контента и управление курсами, тогда как поддержка преподавателя и автоматизация полного образовательного цикла реализованы недостаточно. Существующие AI-решения, такие как Khanmigo, Squirrel AI и универсальные ассистенты на базе ChatGPT, реализуют отдельные функции — чат-боты поддержки, генерацию контента, проверку заданий, — но не обеспечивают комплексного покрытия педагогического цикла в рамках единой системы. Выявленные ограничения существующих платформ — фрагментарность функционала, отсутствие глубокой интеграции в LMS, ограниченная персонализация процесса (а не только контента) и отсутствие системной аналитики — определили требования к разрабатываемой платформе.

Анализ потребностей преподавателей выполнен на основе серии качественных интервью с преподавателями, работающими в различных форматах: индивидуальное обучение взрослых, работа в онлайн-школе с детьми, проведение массовых образовательных программ и марафонов. Применение методологии Jobs To Be Done позволило выделить три ключевых пользовательских сценария — самостоятельный преподаватель, преподаватель в онлайн-школе, эксперт массовых курсов, — а также шесть системных проблем образовательного процесса: фрагментация инструментов и данных, высокая операционная нагрузка, недостаточная автоматизация, отсутствие персонализации на уровне процесса, коммуникационные разрывы, высокая когнитивная и эмоциональная нагрузка. Обобщённая постановка задачи пользователя сформулирована как стремление видеть образовательный процесс единым управляемым целым и получать

поддержку в рутинных задачах при ограниченном времени и разрозненных инструментах. Полученные выводы определили требования к разрабатываемой платформе и обосновали актуальность интеграции AI-ассистента в работу преподавателя как равноправного участника, а не вспомогательного инструмента.

В части экспериментального исследования разработан и протестирован минимально жизнеспособный продукт AI-тьютора на основе многоагентной RAG-архитектуры. В качестве технологического стека обоснован выбор GPT-4o-mini в роли языковой модели, эмбедингов `text-embedding-3-large` размерностью 3072, векторной базы данных Qdrant с алгоритмом HNSW, гибридной системы OCR (Tesseract 5 для однородных отсканированных страниц и мультимодальная VLM через локальный сервер Ollama для страниц со сложной структурой), а также интерфейса в формате Telegram-бота на библиотеке `python-telegram-bot` 20.7. Выбор каждой технологии сопровождается анализом альтернатив: рассмотрены Claude 3.5 Haiku, Llama 3.1 8B и Mistral 7B в качестве LLM, Pinecone, Weaviate и Chroma в качестве векторных хранилищ; для каждой альтернативы зафиксированы причины отклонения по совокупности критериев зрелости, локальной развёртываемости и стоимости инференса.

Реализованный конвейер индексации включает последовательность «Документы → Очистка → Чанкинг → Embedding → Индексация» с автоматической классификацией страниц PDF по типу (цифровая или отсканированная) и сложности содержимого (простой текст, форматированный текст, таблица, смешанная или преимущественно изображение). На основании классификации страницы маршрутизируются между двумя OCR-маршрутами, что обеспечивает баланс качества распознавания и вычислительной стоимости. Реализован двухуровневый кэш эмбедингов и завершений LLM, двухуровневая фильтрация результатов поиска по основному и резервному порогу косинусного сходства, идемпотентный конвейер индексации с миграциями на основе `sentinel`-файлов и атрибуцией каждого извлечённого фрагмента в формате «источник – страница» для последующего цитирования агентом-тьютором.

Экспериментальная оценка проводилась на тестовой выборке из 30 вопросов, разделённых на три группы. На прямых вопросах в терминологии корпуса достигнута точность 90 %, на перефразированных вопросах — 80 %, на вопросах вне корпуса система во всех 10 случаях корректно отказала в ответе с возвращением маркера `[NO_ANSWER]`, не обращаясь к общим знаниям модели; данный результат критически важен с точки зрения педагогической достоверности и предотвращения формирования у студента устойчивых заблуждений на основе галлюцинаций языковой модели. Контрольный эксперимент подтвердил вклад двухуровневой фильтрации: при отключении резервного порога точность по группе перефразированных вопросов снизилась с 90 % до 60 %, что свидетельствует о существенном вкладе резервного порога в полноту ответа без снижения точности. Эмпирический подбор параметров чанкинга показал оптимальность размера окна 400 слов с перекрытием 50 слов (точность 90 % против 73 % при 200 словах и 82 % при 600 словах). Среднее время отклика системы при однопользовательской нагрузке составило 3–5 секунд при доминирующем вкладе генерации ответа GPT-4o-mini (2–4 с), векторизации запроса при промахе кэша (300–500 мс) и пренебрежимо малом вкладе поиска в Qdrant (50–150 мс). Кэширование эмбедингов и завершений ускоряло повторные идентичные запросы практически до мгновенного ответа, что особенно значимо для образовательного сервиса с высокой долей типовых вопросов.

Педагогическое тестирование проведено на шести сценариях, охватывающих основные ветки управляющей логики оркестратора. Агент-планировщик корректно выбирал стратегию (`socratic`, `hint`, `explain`, `verify`) во всех сценариях, что подтвердило отсутствие риска зависания в бесконечном сократическом цикле; в ни одном из тестов система не задавала более двух последовательных наводящих вопросов без перехода к подсказке или объяснению. Точность классификации ответов студента агентом-оценщиком составила 18 из 20 (90 %) на тестовых парах «вопрос – ответ»; диапазоны числовой оценки `score` продемонстрировали корректное ранжирование верных (0,7–1,0), частично верных (0,3–0,7) и неверных (0,0–0,3) ответов; машиночитаемый JSON-вывод корректно парсился во всех случаях, что подтверждает целесообразность принципа структурированного вывода вместо свободного текста при принятии управляющих решений. Тем самым подтверждена гипотеза исследования о том, что интеграция архитектуры Retrieval-Augmented Generation с агентным подходом обеспечивает более высокое качество ответов AI-тьютора по сравнению с использованием только LLM или RAG без агентного слоя; нулевая статистическая гипотеза об отсутствии

различий между архитектурами отклонена в пользу альтернативной. Сравнение базового RAG-ответчика и многоагентного тьютора показало принципиальное различие в педагогической ценности и адаптации к ответам студента при умеренном росте стоимости инференса (в 3,5–4 раза) и средней задержки (с 3–4 до 4–8 секунд).

В части продуктивно-инженерной реализации спроектирована целевая архитектура Smart Line — сквозного AI-помощника студента в платформе Adapstory. Архитектура построена на трёх инженерных принципах: GenAI-native (LLM как основной исполнительный компонент, а не вспомогательный API; каждая функция описывается в терминах роли LLM, механизмов управляемости вывода, контроля знаний и измеримого образовательного эффекта), Plugin-first (минимальное ядро и поставка возможностей через плагины с явной декларацией шаблонов виджетов, допустимых действий и версионированных промптов) и гексагональная декомпозиция доменного слоя, прикладных сценариев и адаптеров со строгим запретом зависимости домена от инфраструктурных фреймворков. Smart Line интегрирован с пятью зонами жизненного цикла студента (Learning, Analytics, Marketplace, Profile, Onboarding), каждой из которых соответствует собственный набор приоритетных намерений и типовых виджетов. Введены три канонические сущности виджета — WidgetIntent, WidgetDraft, WidgetProjection — реализующие принцип «LLM порождает черновики, а не исполняемый интерфейс» и устраняющие класс атак prompt injection на уровне UI за счёт серверной валидации в Policy Gate и сборки итогового DivKit JSON в Projection Builder. Дополнительно введены пять уровней взаимодействия (от L1 Display до L5 Skill Launch), регламентирующих допустимые действия над виджетами и предотвращающих бесконтрольное каскадное обновление проекций.

Семь специализированных агентов (QuestionTypeClassifier, OrganizationalAgent, PlannerAgent, TutorAgent, IntentClassifier, ChatAgent, EvaluatorAgent) реализуют разделение ответственностей в тьюторском конвейере: маршрутизация типа вопроса, генерация педагогического ответа, классификация намерений и оценка ответов студента. Каждый агент инкапсулирует ровно один промпт-шаблон и один тип операции над LLM, что упрощает отладку, замену отдельных компонентов и независимую настройку температуры (низкие значения для классификационных задач, повышенные — для генерации реплик). Введены детерминированные переходы стратегий — принудительный переход к verify после explain, к explain при двух последовательных репликах типа chat, терминальные состояния при достижении мастерства или исчерпании лимита попыток, — обеспечивающие предсказуемость поведения системы при сохранении генеративной природы LLM. Все обращения к языковым моделям маршрутизируются через единый шлюз BC-10 LLM Gateway, выполняющий маскирование персональных данных через механизм PiiMasker, учёт токенов на уровне арендатора со скользящим окном квот, версионирование промптов с семантической версией и SHA-256 отпечатком, валидацию guardrails на минимум десяти положительных и десяти негативных примерах, а также применение паттернов устойчивости (circuit breaker, тайм-ауты, retry с экспоненциальной задержкой). Сквозные события платформы публикуются в Apache Kafka в формате CloudEvents 1.0 с обязательными заголовками идиоматичности, корреляции и трассировки; обработка событий идиоматична за счёт сверки с ProcessedEventStore, упорядоченность на уровне сессии обеспечивается партиционированием по sessionId.

Инфраструктурные требования промышленной эксплуатации сформулированы на основе действующих регламентов разработки и включают: обязательное использование GitOps для развёртывания (запрет ручных операций kubectl apply/edit/patch в продакшн-кластере), подпись образов через Cosign и публикацию SBOM, теги образов в формате {version}-{git-sha} с запретом тега latest, обязательные пробы Kubernetes (liveness, readiness, startup), горизонтальное автомасштабирование с целевой загрузкой CPU 70% и минимумом двух реплик в продакшене, бюджет на корректное завершение по сигналу SIGTERM продолжительностью не менее 30 секунд. Наблюдаемость построена на трёх столпах: сквозной трассировке через OpenTelemetry с экспортом по протоколу OTLP, структурированном логировании в формате JSON с обязательными полями correlation_id, tenant_id, user_id и event, а также метриках SLO/SLI в Prometheus с панелями Grafana, отражающими бюджет ошибок и прогноз его выгорания. Аудит хранится в PostgreSQL с записями о каждом рендере виджета, действии студента, вызове LLM и его стоимости; срок хранения регулируется политикой арендатора. Изоляция арендатора обеспечивается на всех уровнях: HTTP, Redis, Kafka, PostgreSQL, Neo4j и Qdrant — каждый запрос параметризуется tenant_id, что исключает

случайную утечку данных между арендаторами на уровне СУБД, а не только на уровне API.

Реализован MVP-прототип объёмом 1065 строк кода на Python 3.12 и FastAPI 0.136, в который перенесены все семь агентов, RAG-интеграция через Qdrant и пять фабрик виджетов (AskUserQuestion, ExplanationWidget, SmartLineAnswerCard, SkillProgress, InfoCard). Реализованы три HTTP-эндпоинта (/api/health, /api/render, /api/interact) и статический фронтенд с автоматическим переключением между реальным и мок-бэкендом, поддержкой голосового ввода через Web Speech API, требованием ручного подтверждения транскрипта (в соответствии с критерием доступности WCAG 2.1 AA по восстановлению после ошибок) и индикатором текущего режима работы в виде пилули в шапке страницы. Smoke-тестирование подтвердило корректность HTTP-контракта, интеграции с OpenAI и Qdrant, а также воспроизводимость детерминированных переходов стратегий. Двенадцать осознанных ограничений MVP — хранение состояния в памяти процесса, отсутствие LLM Gateway как отдельного слоя, отсутствие маскирования персональных данных перед отправкой в модель, отсутствие эмиссии CloudEvents, отсутствие многоарендности и системы квот, отсутствие операционного мониторинга, использование стандартного логирования вместо JSON, отсутствие проб Kubernetes, отсутствие тестового покрытия и статической типизации, использование Qdrant вместо целевого Neo4j-индекса с графом компетенций, отсутствие Policy Gate и Projection Builder как отдельных компонентов, отсутствие реестра плагинов — явно сопоставлены с компонентами целевой архитектуры, что обеспечивает преемственность между прототипом и промышленной версией без переписывания доменной логики агентов.

Научная новизна работы заключается в разработке концепции образовательной платформы, в которой преподаватель и интеллектуальный AI-агент выступают равноправными участниками образовательного процесса, а не агент — инструментом преподавателя. Предложенная многоагентная RAG-архитектура с явным разделением ответственностей между планировщиком стратегии, генератором реплик и оценщиком ответов студента, а также детерминированными переходами между педагогическими стратегиями, представляет собой методологически обоснованное решение задачи переноса педагогики сократического диалога и конструктивистского подхода в среду больших языковых моделей. Дополнительным элементом новизны является четырёхуровневая система расширяемости платформы — от визуального конструктора без программирования до AI-генерации плагинов, — обеспечивающая совместное участие методистов, разработчиков и AI-агентов в создании образовательных инструментов. Введение трёх канонических сущностей виджета (WidgetIntent, WidgetDraft, WidgetProjection) и пяти уровней взаимодействия (от L1 Display до L5 Skill Launch) формирует контракт между LLM-генерацией и серверной проекцией, позволяющий безопасно использовать языковую модель для динамической композиции пользовательского интерфейса без риска генерации произвольного исполняемого UI на стороне клиента.

Практическая значимость работы определяется тем, что разработанная концепция и MVP-прототип ориентированы на применение в онлайн-школах, образовательных акселераторах и корпоративных программах обучения, реализующих масштабные потоки студентов. Полученные результаты могут быть использованы для разработки и внедрения интеллектуальных образовательных платформ, повышения эффективности организации онлайн-обучения, снижения нагрузки на преподавателей при работе с большими аудиториями и повышения уровня персонализации образовательных программ. Сформированный реестр шаблонов виджетов, контракты HTTP-API и типизированных сообщений Kafka, технологический стек с явным обоснованием альтернатив, а также дорожная карта развития на горизонтах 6, 12 и 24 месяца применимы как непосредственно для проекта Adapstory, так и в качестве референсной архитектуры для аналогичных AI LMS-инициатив. С точки зрения экономической целесообразности применение GPT-4o-mini в качестве основной модели обеспечивает приемлемый баланс качества следования инструкциям и стоимости инференса; кэширование эмбедингов и завершений снижает фактическую стоимость одного хода тьютора при типичной нагрузке в несколько раз, а многоарендный учёт токенов в LLM Gateway позволяет реализовать прозрачное тарифицирование AI-возможностей в коммерческих SaaS-сценариях.

Выявленные ограничения текущей реализации формируют конкретный план дальнейших исследований. К ним относятся: отсутствие персистентного хранения истории учебных сессий, требующее интеграции Redis в качестве StudentStateCache с TTL 1800 секунд и автопродлением; ручная методология оценки, ограничивающая объективность результатов и подлежащая замене на автоматические метрики RAGAS, MRR и

NDCG; примитивный детектор намерений на основе точного вхождения фраз, требующий замены на zero-shot классификацию через языковую модель или специализированный классификатор; отсутствие операционного мониторинга распределения стратегий, доли отказов и среднего балла освоения по темам, без которого невозможно принимать обоснованные решения о доработке системы; ограниченный размер тестовой выборки (30 вопросов для оценки RAG, 6 сценариев для оценки логики оркестратора, 20 пар для оценки агента-оценщика), не позволяющий получить статистически значимые выводы по тонким различиям конфигураций и требующий расширения до сопоставимых с промышленными стандартами объёмов.

Перспективы развития проекта структурированы по четырнадцати направлениям, охватывающим продуктовый, технический, интеграционный и научно-исследовательский домены. К приоритетным задачам ближайшего горизонта (6 месяцев) отнесены: внедрение гибридного RAG с комбинацией BM25, плотного поиска и cross-encoder re-ranker для повышения метрики context precision на 10–15 процентных пунктов и метрики faithfulness на 5–8 процентных пунктов; массовое использование уровня взаимодействия L4 Regeneration в зоне Learning, превращающего Widget Lake из набора независимых карточек в живую поверхность, реагирующую на каждое действие студента; A/B-тестирование промпт-шаблонов с делением трафика арендатора между контрольным и экспериментальным вариантами и автоматическим откатом при деградации ключевых метрик на 10 %; интеграция AI-методиста с AI-тьютором для предвычисления банков проверочных вопросов и контекстов объяснений, что сокращает задержку первого виджета до одной секунды и снижает стоимость холодного старта сессии. Среднесрочный горизонт (12 месяцев) включает мультимодальные расширения (text-to-image-объяснения через модели уровня DALL-E 3 или FLUX, голосовой синтез через Silero для русского языка и ElevenLabs для премиальных сценариев, AI-подкасты по завершении модуля по образцу NotebookLM), локальные конфигурации с Ollama и vLLM на Qwen 2.5 7B для арендаторов с повышенными требованиями к обработке персональных данных и невозможностью использования внешних облачных API, преподавательский дашборд с режимом human-in-the-loop через LangGraph (просмотр трасс сессий, инъекция комментариев в промпт, ручное одобрение шагов в сложных сценариях), а также расширение графа компетенций и адаптивный поиск с учётом зоны ближайшего развития студента. Долгосрочный горизонт (24 месяца) предусматривает supervised fine-tuning базовой модели на корпусе экспертно размеченных диалогов объёмом не менее десяти тысяч пар, мобильное приложение с офлайн-режимом на легковесных моделях уровня Qwen 2.5 0.5B, исполняемых непосредственно на устройстве пользователя, открытие маркетплейса сторонних тьюторских плагинов под управлением BC-02 Plugin Lifecycle с поддержкой подписи артефактов и изоляции промптов, а также проведение лонгитюдных исследований долгосрочной retention обученного материала с измерением сохранности знаний через 4, 8 и 26 недель и сопоставлением различных стратегий интервального повторения (Leitner, Anki-подобная, adaptive).

Особое внимание в дальнейшей работе должно быть уделено вопросам безопасности и этики применения больших языковых моделей в образовательном контексте, особенно при работе с несовершеннолетними учащимися: защите от индукции самоповреждающего поведения, навязывания политических или религиозных позиций, переноса академической нечестности на сторону системы (генерации готового решения вместо наводящего), а также возрастной адаптации лексики и содержательной сложности материала. Каждая из соответствующих политик подлежит оформлению в виде версионированного guardrail с независимым набором тестов в непрерывной интеграции и фиксированным порогом ложноположительных срабатываний; превышение этого порога рассматривается как блокирующий дефект и предотвращает выкат изменения в продакшн. Дополнительным направлением является ранняя детекция выгорания студента через поведенческие признаки (рост числа разговорных реплик, сокращение длины ответов, удлинение пауз, обращения к функции «пропустить») и проактивное предложение смены темы или паузы со стороны специализированного агента-мониторинга мотивации, что перекликается с современными исследованиями в области Affective Tutoring Systems.

Таким образом, в результате выполнения работы разработана научно обоснованная и технически реализуемая концепция образовательной платформы Adapstory с интегрированным AI-тьютором Smart Line; экспериментально подтверждена эффективность многоагентной RAG-архитектуры по сравнению с базовыми подходами на корпусе учебных материалов и тестовой выборке вопросов трёх типов; реализован

функционально замкнутый MVP-прототип объёмом 1065 строк кода, демонстрирующий сквозную работоспособность всех ключевых компонентов; сформирован непротиворечивый план переноса прототипа в промышленную эксплуатацию с явным сопоставлением каждого ограничения MVP с соответствующим компонентом целевой архитектуры; определены приоритетные направления развития на трёхлетнем горизонте с разделением по продуктовым, техническим, интеграционным и научно-исследовательским задачам. Полученные результаты создают методологическую и инженерную основу для построения промышленного AI LMS-решения и подтверждают перспективность интеграции больших языковых моделей в образовательный процесс при условии строгого инженерного контроля управляемости вывода, контроля знаний через RAG и граф компетенций, а также измеримой педагогической ценности каждой возникающей AI-функции. Принципиально, что предложенная архитектура спроектирована не как замкнутое решение конкретной MVP-задачи, а как жизнеспособная платформа для итеративного развития интеллектуального тьютора на горизонте нескольких лет, что подтверждается возможностью добавления новых стратегий обучения, агентов, типов виджетов и моделей без переписывания ядра системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Иванов И. И., Петров П. П. Ансамблевые методы повышения точности классификации данных // Журнал машинного обучения. — 2019. — Vol. 12, no. 3. — P. 15–28.
2. Smith J., Johnson A. Deep Learning Approaches for Data Classification // Journal of Machine Learning Research. — 2020. — Vol. 34, no. 4. — P. 227–246.
3. Goodfellow I., Bengio Y., Courville A. Deep Learning. — Cambridge : MIT Press, 2016.
4. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — 2017. — М.: Стандартинформ, 2017. 32 с.
5. PyTorch Documentation. — дата обращения: 01.03.2025. URL: <https://pytorch.org/docs/stable/index.html>.

Инструкция: Приложение содержит дополнительные материалы, не вошедшие в основной текст работы. В него можно включать изображения, таблицы, программный код, результаты экспериментов и другие вспомогательные данные.